Mohsin Iban Hossain

AIUB, Microprocessor Notes

MICROPROCESSOR

Table of Contents

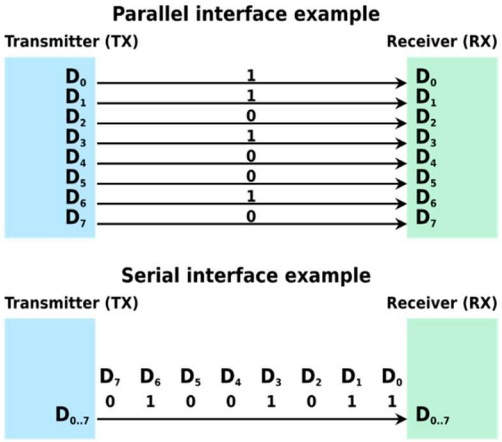
SERIAL COMMUNICATIONS INTERFACES

Data Transmission

⇨ Data transmission is the process of sending and receiving data between two or more devices using a communication medium such as wires, fiber optics, or wireless signals.

Types of Data Transmission

- **Parallel Communication**:
  - Multiple bits sent simultaneously
  - Requires multiple channels
  - Faster but over short distances
- **Serial Communication**:
  - Bits sent one by one
  - Uses a single channel
  - Slower but suitable for long distances



Serial Data Communication

Advantages:

- Fewer communication lines needed than parallel communication
- Only **2 lines** (Tx & Rx) for **asynchronous full-duplex**
- **3 lines** (Tx, Rx & Clock) for **synchronous** communication

Disadvantage:

- Slower data transfer – more time needed compared to parallel communication

Types of Serial Communication

1. **USART (Universal Synchronous/Asynchronous Receiver and Transmitter)**

   ≫ Supports both synchronous and asynchronous data transfer.

2. **SPI (Serial Peripheral Interface)**

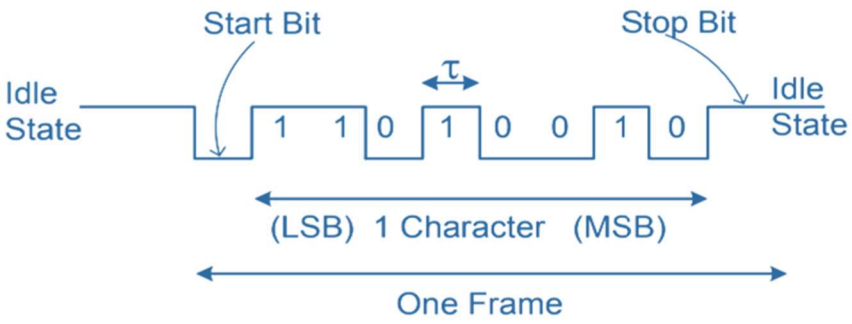
   ≫ High-speed, synchronous, full-duplex communication.

3. **TWI/I2C (Two Wire Interface / Inter-Integrated Circuit)**

   ≫ Synchronous, multi-master communication using only 2 wires.

USART

- It is **asynchronous serial communication**.
- Uses **2 pins from Port D**:
    1. **TXD / PD1** – Transmit data line
    2. **RXD / PD0** – Receive data line
- **Serial Frame Format** for data transmission/reception:
    1. **Start bit** (1 bit)
    2. **Data bits** (5 to 9 bits)
    3. **Optional parity bit** (1 bit)
    4. **Stop bit(s)** (1 or 2 bits)



USART: Internal Clock

USART: Internal Clock Generation – Baud Rate Generator

- Used in **asynchronous** and **synchronous master** modes.
- **UBRRn register** sets the baud rate value.
- A **down-counter** (clocked by $\mathbf{f_{osc}}$) counts down from UBRRn to 0, then reloads.
- This generates the **baud rate clock**.
- **Transmitter** divides this clock by **2, 8, or 16** (based on mode).
- **Receiver** uses the baud rate generator clock **directly** for data sampling and synchronization.

Calculation of the Baud Rate

Operating Mode	Baud Rate Equations	Equations for UBRRn Values
Asynchronous Normal Mode (U2Xn = 0)	$Baud\ rate = \frac{f_{osc}}{16(UBRRn+1)}$	$UBRRn = \frac{f_{osc}}{16 \times Baud\ rate} - 1$
Asynchronous Double Speed Mode (U2Xn = 1)	$Baud\ rate = \frac{f_{osc}}{8(UBRRn+1)}$	$UBRRn = \frac{f_{osc}}{8 \times Baud\ rate} - 1$
Synchronous Master Mode	$Baud\ rate = \frac{f_{osc}}{2(UBRRn+1)}$	$UBRRn = \frac{f_{osc}}{2 \times Baud\ rate} - 1$

Where:

- **Baud rate** = Data transfer rate in **bps**
- $\mathbf{F_{osc}}$ = System clock frequency in **Hz**
- **UBRRn** = Combined value of **UBRRnH** and **UBRRnL** (12 bits total → range: **0–4095)**

Baud Rate

- **Bit-time (τ)**: Time duration of each bit; smaller τ = faster transmission
- **Baud Rate**: Data transfer rate in **bps** (bits per second)
- **Standard baud rates**: 2400, 4800, 9600, 14400, 19200, ...
- In **ATmega328**, baud rate is generated from **internal clock (fosc)**
- **Transmitter and receiver** must use the **same baud rate**
- **Baud error** must be **less than ±2%** to ensure reliable communication

Baud Error Rate Formula:

$$\text{Baud error rate} = \frac{\text{Standard baud rate} - \text{Calculated baud rate}}{\text{Standard baud rate}} \times 100\%$$

Problem1: Find the baud rate for the **three operating modes** when f_{OSC} = 1 MHz and UBRRn = 25. Calculate the baud error and comment whether there will be any communication error or not.

⇨ **is given,**

f_{osc} = 1MHz,

$UBRRn = 25$

For asynchronous normal mode:

⇨ **We Know,**

$$\text{Baud rate} = \frac{f_{osc}}{16(\text{UBRRn} + 1)}$$

$$= \frac{1\times10^6}{16\times(25+1)} = 2404 \text{ bps}$$

$$\text{Baud error rate} = \frac{\text{Standard baud rate} - \text{Calculated baud rate}}{\text{Standard baud rate}} \times 100\%$$

$$= \frac{2400 - 2404}{2400} \times 100\% = -0.167\% < \pm 2\%$$

∴ So, there will be no communication error for the given information.

For asynchronous double speed mode:

⇨ **We Know,**

$$\text{Baud rate} = \frac{f_{osc}}{8(\text{UBRRn} + 1)}$$

$$= \frac{1\times10^6}{8\times(25+1)} = 4808 \text{ bps}$$

$$\text{Baud error rate} = \frac{\text{Standard baud rate} - \text{Calculated baud rate}}{\text{Standard baud rate}} \times 100\%$$

$$= \frac{4800 - 4808}{4800} \times 100\% = -0.167\% < \pm 2\%$$

∴ So, there will be no communication error for the given information.

For synchronous master mode:

⇨ **We Know,**

$$\text{Baud rate} = \frac{f_{osc}}{2(\text{UBRRn} + 1)}$$

$$= \frac{1\times10^6}{2\times(25+1)} = 19231 \text{bps}$$

$$\text{Baud error rate} = \frac{\text{Standard baud rate} - \text{Calculated baud rate}}{\text{Standard baud rate}} \times 100\%$$

$$= \frac{19200 - 19231}{91200} \times 100\% = -0.161\% < \pm 2\%$$

∴ So, there will be no communication error for the given information.

Problem2: Find the **baud rate** for the **synchronous operating mode** when the oscillator frequency, **fosc= 16 MHz,** and register data is, **UBRRn = 101010111001**. Calculate the **baud error and comment** on whether there will be any communication error or not.
For Arduino Uno, standard Baud rates maybe 300, 1200, 2400,3000, 4800, 9600, 19200, 38400, 57600, 115200, etc.

⇨ **is given,**

f_{osc} = 16MHz,

$$UBRRn = \underbrace{\mathbf{101010111001}}_{(in\ binary)} = \underbrace{\mathbf{2745}}_{(in\ deciaml)}$$

For synchronous master mode:

⇨ **We Know,**

$$\textbf{Baud rate} = \frac{f_{osc}}{2(\textbf{UBRRn} + 1)}$$

$$= \frac{16\times10^6}{2\times(2745+1)} = \mathbf{2914}\ \textbf{bps}$$

Compare with Standard Baud Rates

⇨ Closest standard baud rate: **3000 is not standard**, but **2400** and **4800** are nearby.
Use 2400 bps as the **nearest lower standard**.

$$\textbf{Baud error rate} = \frac{\textbf{Standard baud rate} - \textbf{Calculated baud rate}}{\textbf{Standard baud rate}} \times 100\%$$

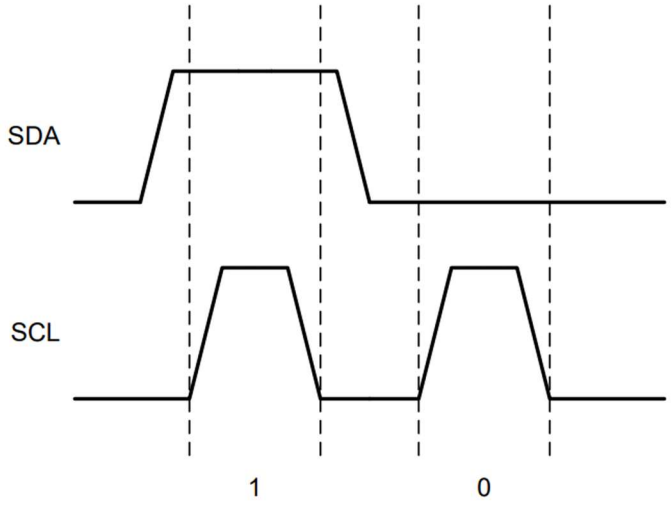
$$= \frac{2400 - 2914}{2400} \times 100\%$$

$$= -21.416\% > \pm 2\%$$

∴ **So, there will be communication error for the given information due to high baud mismatch.**

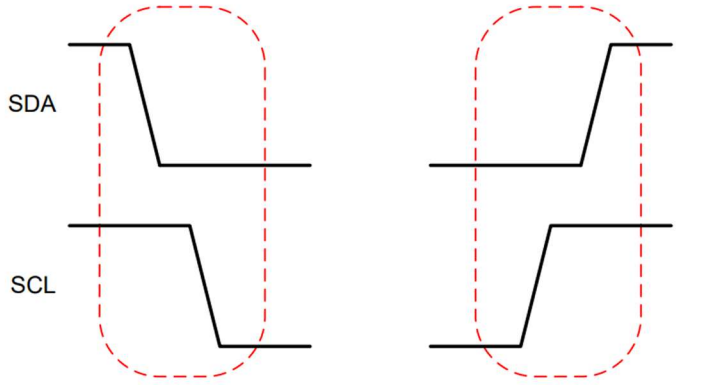
I2C (INTER-INTEGRATED CIRCUIT)

I2C Digital One and Zero Representations

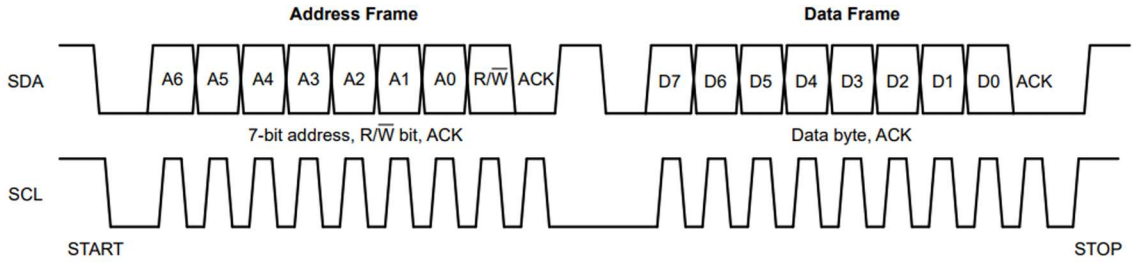


Working of I2C in Arduino

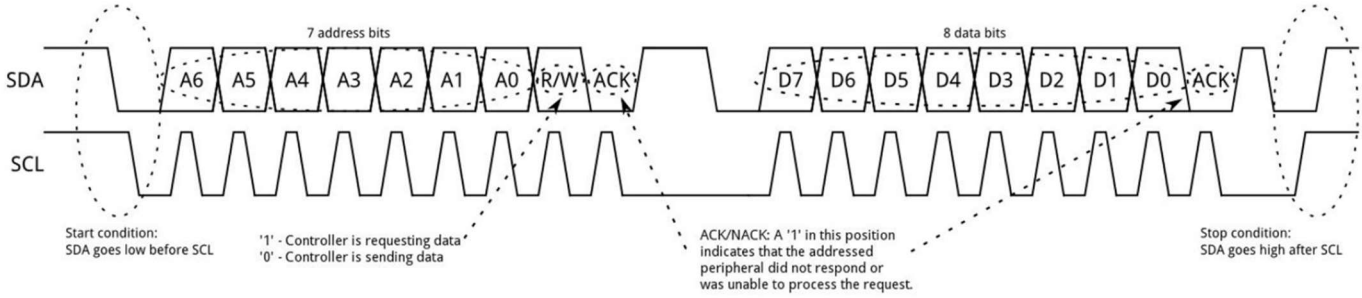
- **Start Condition:** The SDA line switches from **high to low** voltage level before SCL switches from **high to low.**
- **Stop Condition:** The SDA line switches from **low to high** voltage level after SCL switches from **low to high**.



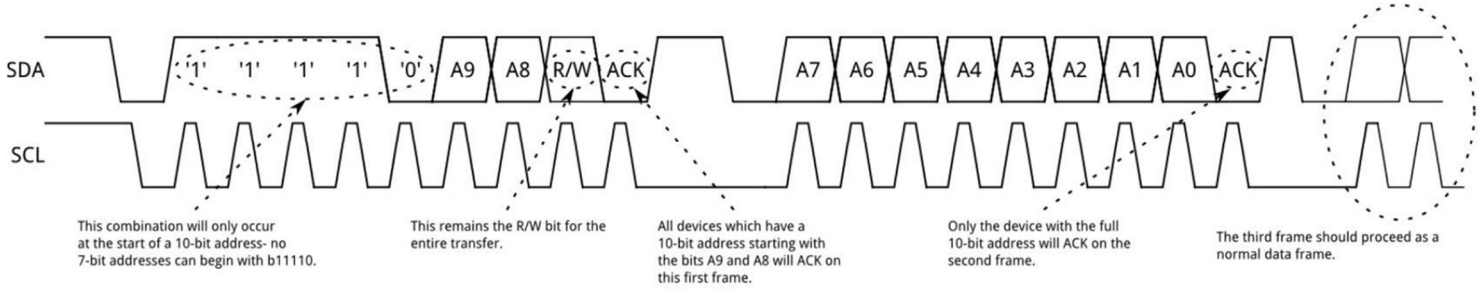
I2C Address and Data Frames



7-bit Addressing



10-bit Addressing

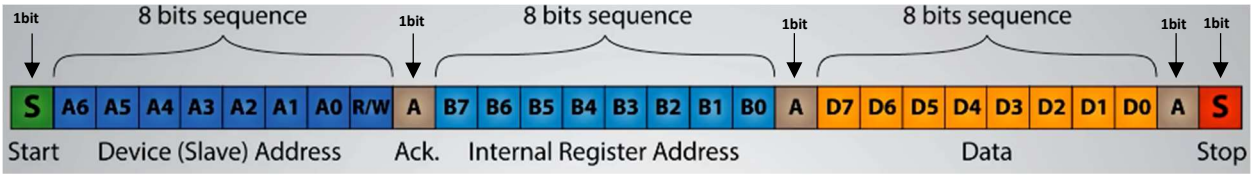


Problem1:

(a) If The **I²C SCL (clock) frequency is 1MHz** Based on the I²C protocol shown in the table, calculate the total **frame rate**, **frame duration** (in microseconds) for the complete write sequence.

Solution:

(a)



∴ **Total bits** transmitted = 1 (Start) + 8 + 1 + 8 + 1 + 8 + 1 + 1 (Stop) = **29 bits**

We Know,

$$\textbf{Frame Rate} = \frac{\textbf{Clock Frequency (SCL)}}{\textbf{Total Number of Bits}}$$

$$\textbf{Frame Rate} = \frac{1M}{29} = 34.48k\ fps$$

$$\textbf{Frame Duration} = \frac{1}{\textbf{Frame Rate}} = \frac{1}{34.48k} = 29\mu s$$

Problem2:

S	Target Address	W	A	Target Address	A	Sr	Target Address	R	A	Data	Ā	P
0	11110XX	0	0	XXXXXXX0	0	0	11110XX	1	0	DATA	1	1

(a) If The **I²C SCL (clock) frequency is 1MHz** Based on the I²C protocol shown in the waveform, calculate the total **frame rate** for the complete write sequence.

Solution:

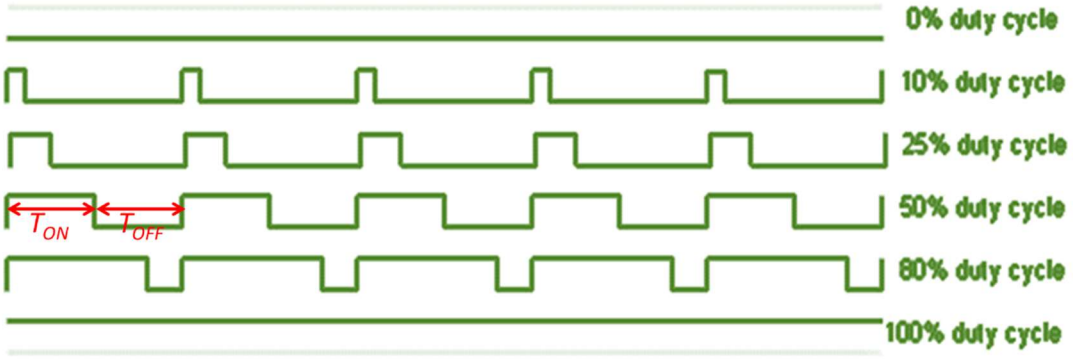
∴ **Total bits** transmitted = 1 (Start) + 7+ 1 + 1 + 8 + 1 +1 + 7 + 1 + 1 + 8 + 1 + 1 (Stop) = **39 bits**

$$\text{rame Rate} = \frac{\textbf{Clock Frequency (SCL)}}{\textbf{Total Number of Bits}} = \frac{1M}{39} = 25.64k\ fps$$

PULSE WIDTH MODULATION (PWM)

Pulse Width Modulation (PWM)

- **PWM** simulates **analog output** using a **digital signal** (ON/OFF square wave).
- The **"on time"** in each cycle is called the **pulse width**.
- Varying the **pulse width** changes the **average voltage**.
- On an **Arduino UNO**, output ranges between **0V and 5V**.
- When repeated fast enough, it creates the effect of a **smooth analog output** (e.g., controlling LED brightness or motor speed).



Duty Cycle

- Definition: Ratio of ON time to total time period of the PWM signal
- **Formula**:

$$\mathbf{D = \frac{T_{ON}}{T} \times 100\%}$$

- Where:
  - T_{ON} = Duration signal is **ON**
  - T_{OFF}= Duration signal is **OFF**
  - $\mathbf{T = T_{ON} + T_{OFF} =}$ **Total period**
- **Higher duty cycle** → More time ON → **Higher average voltage**
- **Lower duty cycle** → More time OFF → **Lower average voltage**

PWM Modes

➪ **Main PWM Modes:**

- **Fast PWM**
- **Phase-Correct PWM**

Fast PWM Modes (Timer/Counter 0):

- **Mode 3** and **Mode 7**
- Selected using **WGM02**, **WGM01**, and **WGM00** bits
  - **WGM01 & WGM00** → in **TCCR0A**
  - **WGM02** → in **TCCR0B**

Bit	7	6	5	4	3	2	1	0	
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	$clk_{I/O}$/(No prescaling)
0	1	0	$clk_{I/O}$/8 (From prescaler)
0	1	1	$clk_{I/O}$/64 (From prescaler)
1	0	0	$clk_{I/O}$/256 (From prescaler)
1	0	1	$clk_{I/O}$/1024 (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

➢ **Fast PWM Types:**

1. **Non-Inverting Mode**: Output is HIGH during ON time
2. **Inverting Mode**: Output is LOW during ON time

These modes define how the output signal behaves relative to the counter's compare match

Difference Between Fast PWM Modes 3 and 7

- **Mode 3 (Fast PWM)**
  - **TOP value**: Fixed at **0xFF (255)**
  - Auto wraps from **0 to 255**
  - No need to set **OCRnA** for TOP
- **Mode 7 (Fast PWM)**
  - **TOP value**: Set by **OCRnA**
  - User must load **OCRnA** with desired TOP value
  - Allows flexible frequency and resolution
- In both modes, the timer counts **from BOTTOM to TOP**, then restarts from BOTTOM continuously.

Mode	WGM02	WGM01	WGM00	Timer/Counter Mode of Operation	TOP	Update of OCRx at	TOV Flag Set on(1)(2)
0	0	0	0	Normal	0xFF	Immediate	MAX
1	0	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM
2	0	1	0	CTC	OCRA	Immediate	MAX
3	0	1	1	Fast PWM	0xFF	BOTTOM	MAX
4	1	0	0	Reserved	–	–	–
5	1	0	1	PWM, Phase Correct	OCRA	TOP	BOTTOM
6	1	1	0	Reserved	–	–	–
7	1	1	1	Fast PWM	OCRA	BOTTOM	TOP

Bit	7	6	5	4	3	2	1	0	
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Fast PWM Mode: Setting Modes

⇨ To select **non-inverting** or **inverting** Fast PWM waveforms, configure the **Compare Output Mode** bits in **TCCR0A** register:

- **For OC0A (COM0A1, COM0A0)**
- **For OC0B (COM0B1, COM0B0)**

➢ Fast PWM Waveform Mode Selection:

Mode	COM0x1	COM0x0	Description
Normal (disconnected)	0	0	OC0x disconnected
Non-Inverting	1	0	Clear OC0x on compare match, set at BOTTOM
Inverting	1	1	Set OC0x on compare match, clear at BOTTOM

☑ Use **Non-Inverting** for increasing duty cycle → increasing output

☑ Use **Inverting** for increasing duty cycle → decreasing output

When **WGM02:0** bits are set for **Fast PWM mode**, the **COM0A1:0** bits in the **TCCR0A** register control the behavior of the **OC0A** pin:

Table 12-3. Compare Output Mode, Fast PWM Mode(1)

COM0A1	COM0A0	Description
0	0	Normal port operation, OC0A disconnected.
0	1	WGM02 = 0: Normal Port Operation, OC0A Disconnected. WGM02 = 1: Toggle OC0A on Compare Match.
1	0	Clear OC0A on Compare Match, set OC0A at BOTTOM, (non-inverting mode).
1	1	Set OC0A on Compare Match, clear OC0A at BOTTOM, (inverting mode).

When **WGM02:0** bits are set for **Fast PWM mode**, the **COM0B1:0** bits in the **TCCR0B** register control the behavior of the **OC0B** pin:

Table 12-6. Compare Output Mode, Fast PWM Mode(1)

COM0B1	COM0B0	Description
0	0	Normal port operation, OC0B disconnected.
0	1	Reserved
1	0	Clear OC0B on Compare Match, set OC0B at BOTTOM, (non-inverting mode)
1	1	Set OC0B on Compare Match, clear OC0B at BOTTOM, (inverting mode).

Note: 1. A special case occurs when OCR0B equals TOP and COM0B1 is set. In this case, the Compare Match is ignored, but the set or clear is done at TOP. See "Fast PWM Mode" on page 101 for more details.

Notes:

- The **Toggle** mode (`01`) is **only available for OC0A, not OC0B**.
- For PWM:
  - Use `10` for **Non-inverting** (default and commonly used).
  - Use `11` for **Inverting** (used when active-low control is needed).

Fast PWM Mode Overview

⇨ **Waveform Generation Mode (WGM02:0 = 3 or 7)**

⇨ **Single-slope operation**:

Counter counts **from BOTTOM to TOP**, then **resets to BOTTOM**.

➢ Output Behavior:

Non-Inverting Mode (COM0x1:0 = 10):

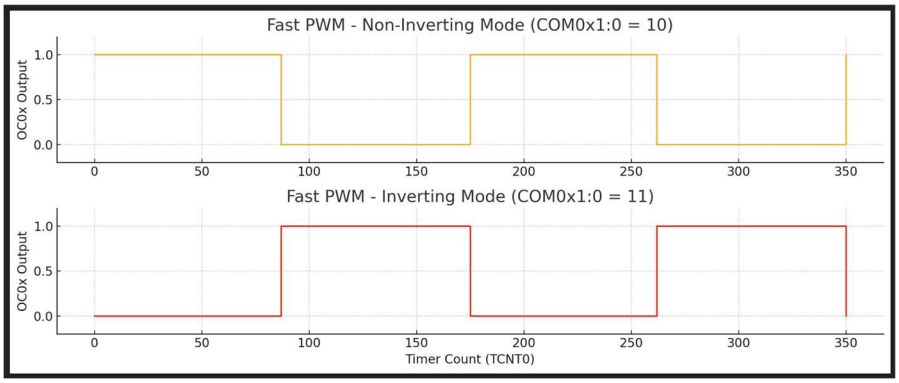
- **OC0x is cleared** on compare match (`TCNT0 == OCR0x`)
- **OC0x is set** at **BOTTOM**

Inverting Mode (COM0x1:0 = 11):

- **OC0x is set** on compare match (`TCNT0 == OCR0x`)
- **OC0x is cleared** at **BOTTOM**

This difference causes:

- **Non-Inverting PWM** to start HIGH and go LOW at match.
- **Inverting PWM** to start LOW and go HIGH at match.



Fast PWM Mode: Output Frequency

1. OC0x Visibility

The PWM waveform will **only appear** on the `OC0A` or `OC0B` pin **if** the pin is configured as an output:

```
    DDRD |= (1 << PD5); // OC0A (PD5) set as output
```

2. Output Waveform Behavior (Fast PWM)

Mode	OC0x Behavior
Non-inverting	**Clear** on Compare Match, **Set** at BOTTOM (TCNT0 = 0)
Inverting	**Set** on Compare Match, **Clear** at BOTTOM (TCNT0 = 0)

3. Frequency Formulas

- Mode 3: Fast PWM (TOP = 0xFF = 255)
  - **PWM frequency (non-inverted/inverted):**

$$f_{OCnx(FPWM)} = \frac{f_{clk\_IO}}{N \times 256}$$

- Mode 7: Fast PWM (TOP = OCR0A)

$$f_{OCnx(FPWM)} = \frac{f_{clk\_IO}}{N \times (1 + OCRnx)}$$

Where:

- $f_{clk\_IO} = System\ Clock\ (e.g., 16\ MHz\ for\ Arduino\ Uno)$
- $N = Prescaler\ (1, 8, 64, 256, or\ 1024)$

4. Maximum PWM Frequency

➢ **If `OCR0A = 0` (Mode 7), then:**

$$f_{OC0x} = f_{clk\_IO}$$

Non-Inverting Fast PWM Duty Cycle Formula	
When `TOP = 0xFF` (**Mode 3** - Fast PWM with TOP fixed)	When `TOP = OCR0A` (**Mode 7** - Fast PWM with variable TOP)
The duty cycle **D** (%) relates to **OCR0x** as: $$\mathbf{OCR0x} = \left(\frac{256 \times D}{100}\right) - 1$$ ⇨ `x` = **A or B** ⇨ `D` = **desired duty cycle in %** ⇨ **Result loaded to `OCR0x`**	The duty cycle for **OCR0B** output is: $$\mathbf{OCR0B} = \left(\frac{(1 + \mathbf{OCR0A}) \times D}{100}\right) - 1$$

Example1: Non-Inverting, Compute `OCR0A` for 75% duty, TOP = 0xFF

⇨ **is given,**

$D = 75\%,$

$\text{TOP} = 0\text{xFF}$

⇨ **We Know,**

$$\mathbf{OCR0A} = \left(\frac{256 \times D}{100}\right) - 1$$

$$= \frac{256 \times 75}{100} - 1 = 191$$

Example2: Non-Inverting PWM with 75% duty if `OCR0A = 200`, Compute `OCR0B`

⇨ **is given,**

$D = 75\%,$

$\text{OCR0A} = 200$

⇨ **We Know,**

$$\mathbf{OCR0B} = \left(\frac{(1 + \mathbf{OCR0A}) \times \boldsymbol{D}}{\mathbf{100}}\right) - \mathbf{1}$$

$$= \frac{(1+200)\times 75}{100} - 1 = 149.75 \approx 150$$

Inverting Fast PWM Duty Cycle Formula	
When `TOP = 0xFF` (**Mode 3** - Fast PWM with TOP fixed)	When `TOP = OCR0A` (**Mode 7** - Fast PWM with variable TOP)
The duty cycle **D** (%) relates to **OCR0x** as: $\mathbf{OCR0x} = 255 - \left(\frac{256 \times \boldsymbol{D}}{\mathbf{100}}\right)$ ⇨ `x` = A or B ⇨ `D` = desired duty cycle in % ⇨ Result loaded to `OCR0x`	The duty cycle for **OCR0B** output is: $\mathbf{OCR0B} = \mathbf{OCR0A} - \left(\frac{(1 + \mathbf{OCR0A}) \times \boldsymbol{D}}{\mathbf{100}}\right)$

Example3: Inverting PWM with 75% duty, TOP = 0xFF, Compute `OCR0B`

⇨ **is given,**

$D = 75\%,$

$\text{TOP} = 0\text{xFF}$

⇨ **We Know,**

$$\mathbf{OCR0B} = 255 - \left(\frac{256 \times \boldsymbol{D}}{\mathbf{100}}\right)$$

$$= 255 - \frac{256\times 75}{100} = 63$$

Example4: Inverting PWM with 75% duty, `OCR0A = 200`, compute `OCR0B`

⇨ **is given,**

$D = 75\%$,

$\text{OCR0A} = 200$

⇨ **We Know,**

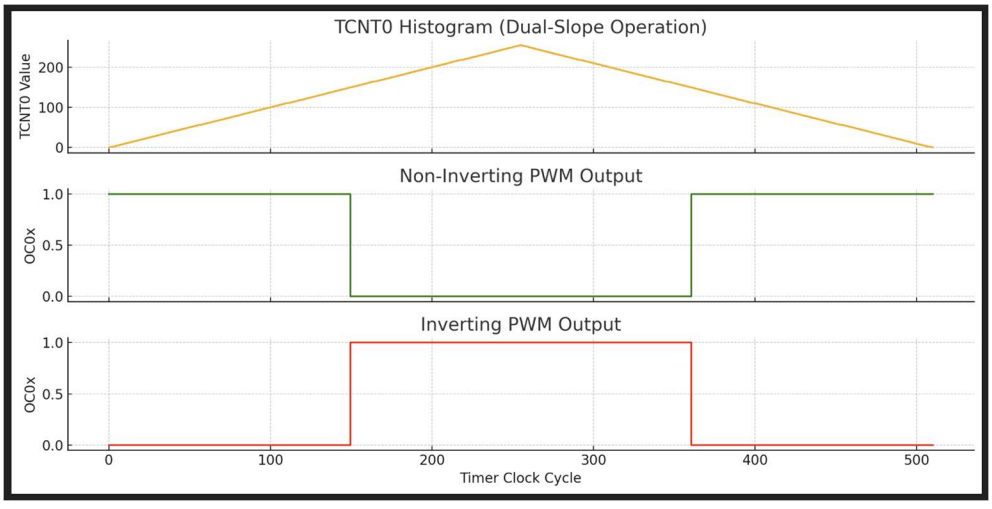
$$\mathbf{OCR0B = OCR0A - \left(\frac{(1 + OCR0A) \times D}{100}\right)}$$

$$\mathbf{= 200 - \frac{(1+200)\times 75}{100} = 49.25 \approx 49}$$

Phase Correct PWM Mode

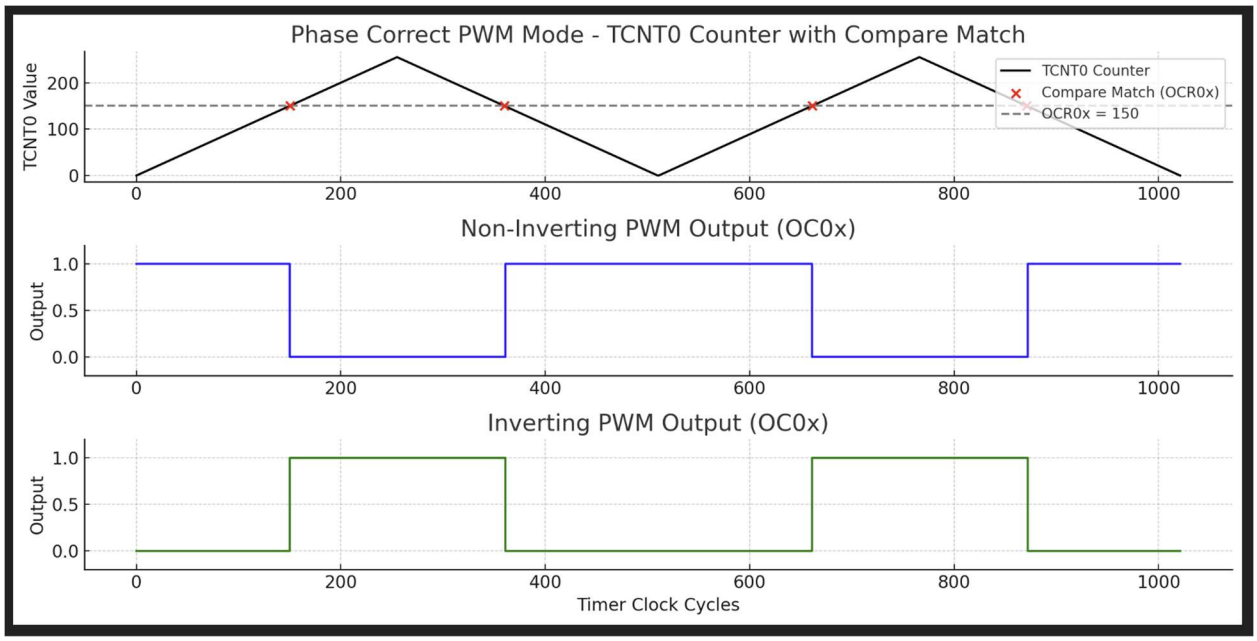
- **Dual-slope operation**:
  - Counter counts **BOTTOM → TOP → BOTTOM**
- More **symmetrical waveform** than Fast PWM
- Lower **maximum frequency** than Fast PWM
- Preferred in applications needing symmetry (e.g., **motor control**)

Timing diagram for Phase Correct PWM Mode



Here is the:

1. **Top plot**: The TCNT0 counter value follows a **dual-slope operation** — it counts from 0 to TOP (255) and back down to 0, repeating.
2. **Middle plot**: Shows the **Non-Inverting PWM output** — it is high when TCNT0 < OCR (150) and goes low during the rest.
3. **Bottom plot**: Shows the **Inverting PWM output** — it is the inverse of the non-inverting output.



This diagram illustrates **Phase Correct PWM Mode** for both **Non-Inverting** and **Inverting** outputs:

- The **first plot** shows the `TCNT0` counter behavior (dual-slope: up to TOP and down to BOTTOM).
- The **red dots** mark the compare matches when **`TCNT0 == OCR0x.`**
- The **second plot** is the **non-inverting PWM output** — it clears (goes low) on compare match while counting up and sets (goes high) on compare match while counting down.
- The **third plot** is the **inverting PWM output** — the opposite behavior of non-inverting.

Phase Correct PWM Mode: Setting Modes

➔ When the **WGM02:0 bits are set to 1 or 5** (Phase Correct PWM modes), the behavior of **OC0x** outputs depends on these bits as outlined in **Table 12-7** (from the datasheet):

COM0x1	COM0x0	Description
0	0	Normal port operation, OC0x disconnected
0	1	Toggle OC0x on compare match (only OC0A, and only when WGM02 = 1)
1	0	Clear OC0x on compare match when up-counting, set on down-counting (non-inverting)
1	1	Set OC0x on compare match when up-counting, clear on down-counting (inverting)

Table 12-7. Compare Output Mode, Phase Correct PWM Mode(1)

COM0B1	COM0B0	Description
0	0	Normal port operation, OC0B disconnected.
0	1	Reserved
1	0	Clear OC0B on Compare Match when up-counting. Set OC0B on Compare Match when down-counting.
1	1	Set OC0B on Compare Match when up-counting. Clear OC0B on Compare Match when down-counting.

Notes:

- The **Toggle** mode (`01`) is **only available for OC0A, not OC0B**.
- For PWM:
  - Use `10` for **Non-inverting** (default and commonly used).
  - Use `11` for **Inverting** (used when active-low control is needed).

Phase Correct PWM Mode Overview

- **Dual-slope operation**:
  - Counter counts **BOTTOM → TOP → BOTTOM**
- More **symmetrical waveform** than Fast PWM
- Lower **maximum frequency** than Fast PWM

➢ **OC0x behavior:**

⇨ **Cleared/Set** on compare match (OCR0x = TCNT0) during **increment**.

⇨ **Set/Cleared** on compare match (OCR0x = TCNT0) during **decrement**.

Phase Correct PWM Mode: Output Frequency

1. OC0x Visibility

The PWM waveform will **only appear** on the `OC0A` or `OC0B` pin **if** the pin is configured as an output:

```
    DDRD |= (1 << PD5); // OC0A (PD5) set as output
```

2. Output Waveform Behavior (Phase Correct PWM)

Mode	OC0x Behavior
Non-inverting	**Clear on Compare Match** when counting **up**, **Set on Compare Match** when counting **down**
Inverting	**Set on Compare Match** when counting **up**, **Clear on Compare Match** when counting **down**

4. Frequency Formulas

◇ Mode 1: Phase Correct PWM (TOP = 0xFF = 255)

$$f_{OCnx(PCPWM)} = \frac{f_{clk\_IO}}{N \times 510}$$

◇ Mode 5: Phase Correct PWM (TOP = $OCRnx$):

$$f_{OCnx(PCPWM)} = \frac{f_{clk\_IO}}{2N \times (OCRnx)}$$

Where:

- $f_{clk\_IO} = System\ Clock\ (e.g., 16\ MHz\ for\ Arduino\ Uno)$
- $N = Prescaler\ (1, 8, 64, 256, or\ 1024)$

4. Maximum PWM Frequency

➢ **If `OCR0A` = 0 (Mode 7), then:**

$$f_{OC0x(Max)} = f_{clk\_IO}$$

OCR0A Extreme Values (Phase Correct PWM)

- **Non-Inverting Mode**:
  - **OCR0A = BOTTOM (0)** → Output = **Always LOW**
  - **OCR0A = MAX (255)** → Output = **Always HIGH**
- **Inverting Mode**:
  - **OCR0A = BOTTOM (0)** → Output = **Always HIGH**
  - **OCR0A = MAX (255)** → Output = **Always LOW**

Phase Correct PWM Mode Setup

Mode	WGM02	WGM01	WGM00	Timer/Counter Mode of Operation	TOP	Update of OCRx at	TOV Flag Set on(1)(2)
0	0	0	0	Normal	0xFF	Immediate	MAX
1	0	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM
2	0	1	0	CTC	OCRA	Immediate	MAX
3	0	1	1	Fast PWM	0xFF	BOTTOM	MAX
4	1	0	0	Reserved	–	–	–
5	1	0	1	PWM, Phase Correct	OCRA	TOP	BOTTOM
6	1	1	0	Reserved	–	–	–
7	1	1	1	Fast PWM	OCRA	BOTTOM	TOP

Difference Between Fast PWM Modes 3(1) and 7(5)

- **Mode 3(1)**:
  - **TOP = 0xFF (Fixed)**
- **Mode 7(5)**:
  - **TOP = OCRnA (Variable)**
  - Must **load OCRnA** with desired count value
- **Behavior (Both Modes)**:
  - Counter counts **from BOTTOM to TOP**, then **restarts from BOTTOM**

Example5: Calculate the PWM frequency for the output when using Fast PWM Mode and Phase Correct PWM Mode when f_{clk_IO} is **10 MHz** and the pre-scale factors are **1, 8, 64, 256,** or **1024.** Comment on the results afterward.

⇨ **is given,**

f_{clk_IO} = 10MHz

ps = 1024

For Fast PWM Mode:

⇨ **We Know,**

$$f_{OCnx(FPWM)} = \frac{f_{clk_{IO}}}{N \times 256}$$

$$= \frac{10 \times 10^6}{1024 \times 256} = 38.15Hz$$

For Phase Correct PWM Mode:

⇨ **We Know,**

$$f_{OCnx(PCPWM)} = \frac{f_{clk_{IO}}}{N \times 510}$$

$$= \frac{10 \times 10^6}{1024 \times 510} = 19.15Hz$$

∴ **Fast PWM is ~2 × faster than Phase Correct PWM**

Phase Correct PWM Mode: Output Frequency

Non-Inverting Phase Correct PWM Duty Cycle Formula	
When `TOP = 0xFF` (**Mode 1** - Fast PWM with TOP fixed)	When `TOP = OCR0A` (**Mode 5** - Fast PWM with variable TOP)
The duty cycle **D** (%) relates to **OCR0x** as: $$\mathbf{OCR0x} = \left(\frac{\mathbf{255} \times \boldsymbol{D}}{\mathbf{100}}\right)$$ ⇨ **x = A or B** ⇨ **D = desired duty cycle in %** ⇨ **Result loaded to OCR0x**	The duty cycle for **OCR0B** output is: $$\mathbf{OCR0B} = \left(\frac{(\mathbf{OCR0A}) \times \boldsymbol{D}}{\mathbf{100}}\right)$$

Example6: Non-Inverting with 75% duty, TOP = 0xff, Compute `OCR0A`

⇨ **is given,**

$D = 75\%,$

$\text{TOP} = 0\text{xFF}$

⇨ **We Know,**

$$\mathbf{OCR0A} = \left(\frac{\mathbf{255} \times \boldsymbol{D}}{\mathbf{100}}\right)$$

$$= \frac{255 \times 75}{100} = \mathbf{191.25}$$

Example7: Non-Inverting PWM with 75% duty if `OCR0A = 200`, Compute `OCR0B`

⇨ **is given,**

$D = 75\%,$

$\text{OCR0A} = 200$

⇨ **We Know,**

$$\mathbf{OCR0B} = \left(\frac{(\mathbf{OCR0A}) \times \boldsymbol{D}}{\mathbf{100}}\right)$$

$$= \frac{(200) \times 75}{100} = \mathbf{150}$$

Inverting Fast PWM Duty Cycle Formula

When `TOP = 0xFF` (**Mode 3** - Fast PWM with TOP fixed)	When `TOP = OCR0A` (**Mode 7** - Fast PWM with variable TOP)
The duty cycle **D** (%) relates to **OCR0x** as: $$\mathbf{OCR0x = 255 - \left(\frac{255 \times D}{100}\right)}$$ ⇨ `x` = A or B ⇨ `D` = desired duty cycle in % ⇨ Result loaded to `OCR0x`	The duty cycle for **OCR0B** output is: $$\mathbf{OCR0B = OCR0A - \left(\frac{(OCR0A) \times D}{100}\right)}$$

Example8: Inverting PWM with 75% duty, TOP = 0xFF, Compute `OCR0B`

⇨ **is given,**

$D = 75\%$,

$\text{TOP} = 0\text{xFF}$

⇨ **We Know,**

$$\mathbf{OCR0B = 255 - \left(\frac{255 \times D}{100}\right)}$$

$$\mathbf{= 255 - \frac{256 \times 75}{100} = 63.75}$$

Example9: Inverting PWM with 75% duty, `OCR0A = 200`, compute `OCR0B`

⇨ **is given,**

$D = 75\%$,

$\text{OCR0A} = 200$

⇨ **We Know,**

$$\mathbf{OCR0B = OCR0A - \left(\frac{(OCR0A) \times D}{100}\right)}$$

$$\mathbf{= 200 - \frac{(200) \times 75}{100} = 50}$$

Example10: Use ATmega328p **Timer 0 Fast PWM mode 7** with TOP at OCR0A and toggle on **OC0A pin (PD06)**. Calculate the PWM frequency. The frequency changes with OCR0A values (100 and 200) loaded. **Calculate** the fast PWM frequency. The Pre-scaler value, *N* = 1, and the system clock frequency, f_{clk_IO} = 16 MHz.

⇨ **is given,**

$Mode = 7$, $f_{clk\_IO} = 16\text{MHz}$,

$\text{OCR0A} = 100 \;\&\; 200$ $N = 1$

For OCR0A = 100:

⇨ **We Know,**

$$f_{OCR0A\,(FPWM)} = \frac{f_{clk_{IO}}}{N \times (1 + OCR0x)}$$

$$= \frac{16 \times 10^6}{1 \times (1 + 100)} = 158.41\text{K Hz}$$

For OCR0A = 200:

⇨ **We Know,**

$$f_{OCR0A\,(FPWM)} = \frac{f_{clk_{IO}}}{N \times (1 + OCRnx)}$$

$$= \frac{16 \times 10^6}{1 \times (1 + 200)} = 79.60\text{K Hz}$$

Programming Arduino for Fast PWM

For Mode 3: Fast PWM (TOP = 0xFF = 255):

```
#ifndef F_CPU
#define F_CPU 16000000UL      // Clock frequency is 16 MHz
#endif
#include <avr/io.h>

int main() {
// OC0B pin is set as PWM output pin by sending a HIGH to Port D's Data Direction Register, DDRD
DDRD |= (1<<PD5);
OCR0B = 191; // Load 191 into OCR0B for setting its duty cycle to 75% (See previous example, Example:1*
// Configure TCCR0A and TCCR0B register for (i) non-inverting (10), (ii) Fast PWM mode 3 (011),
// (iii) Pre-scalar (101) for frequency control. By default, 0s are there, but you may set them explicitly.
TCCR0A |= (1 << COM0B1) | (1<<WGM01) | (1<<WGM00);
TCCR0B |= (1<<CS02) | (1<<CS00);  // CS02:00 = 101, N = 1024
// TCCR0A=0b00100011; TCCR0B=0b00000101
while(1);
return 0;
}
```

Fast PWM Generated Waveform

For Mode 3: Fast PWM (TOP = 0xFF = 255):

⇨ **If,**

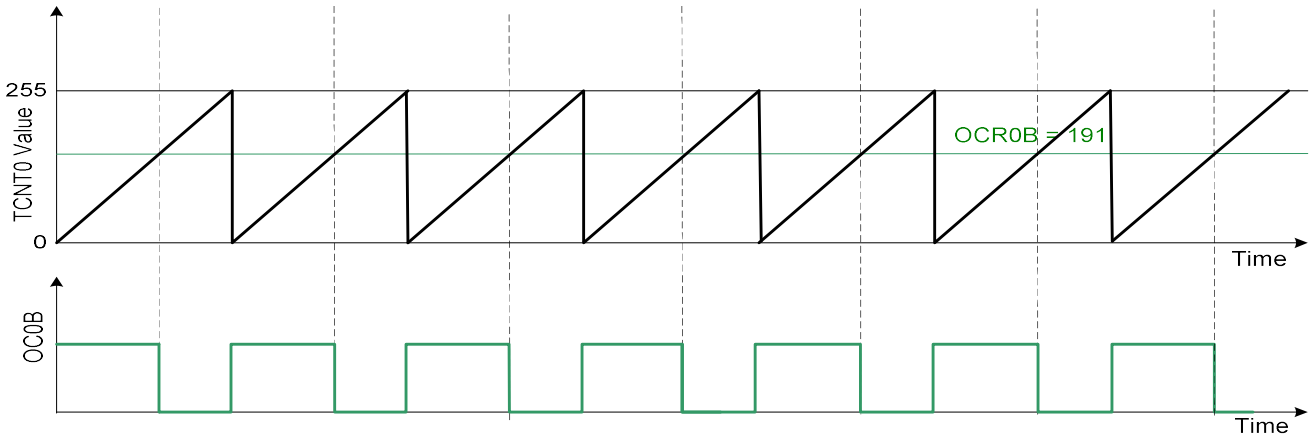
$Mode = 3$, $f_{clk\_IO} = 16\text{MHz}$,

$N = 1024$, OCR0B = 191

⇨ **We Know,**

$$f_{OCnx(FPWM)} = \frac{f_{clk\_IO}}{N \times 256}$$

$$f_{OCR0B(FPWM)} = \frac{16 \times 10^6}{1024 \times 256} = 61.03Hz$$



How the Waveform is Generated:

- The timer starts from 0, counts up to 255.
- Each time TCNT0 equals OCR0B (191), the **OC0B pin is cleared (goes LOW)**.
- When TCNT0 overflows back to 0, **OC0B is set (goes HIGH)** again.
- This results in a PWM waveform with:
  - **High time** from 0 to 191 counts
  - **Low time** from 191 to 255 counts

Programming Arduino for Fast PWM

For Mode 7: Fast PWM (TOP = OCR0A):

```
#ifndef F_CPU
#define F_CPU 8000000UL   // Clock frequency is 8 MHz
#endif
#include <avr/io.h>

int main() {
// OC0B pin is set as PWM output pin by sending a HIGH to Port D's Data Direction Register, DDRD
DDRD |= (1<<PD5);
OCR0A = 200;    // Top Value of 200 (must be equal or greater than the OCR0B)
OCR0B = 140; // Load 140 into OCR0B for setting its duty cycle to 70% (See previous example, Example:2*)
// Configure TCCR0A and TCCR0B register for (i) non-inverting (10), (ii) Fast PWM mode 7 (111),
// (iii) Pre-scalar (011) for frequency control. By default, 0s are there, but you may set them explicitly.
TCCR0A |= (1 << COM0B1) | (1<<WGM01) | (1<<WGM00);
TCCR0B |= (1<<WGM02) | (1<<CS01) | (1<<CS00);  // CS02:00 = 011, N = 64
// TCCR0A=0b00100011; TCCR0B=0b00001011
while(1);
return 0;
}
```

Fast PWM Generated Waveform

For Mode 7: Fast PWM (TOP = OCR0A):

⇨ **If,**

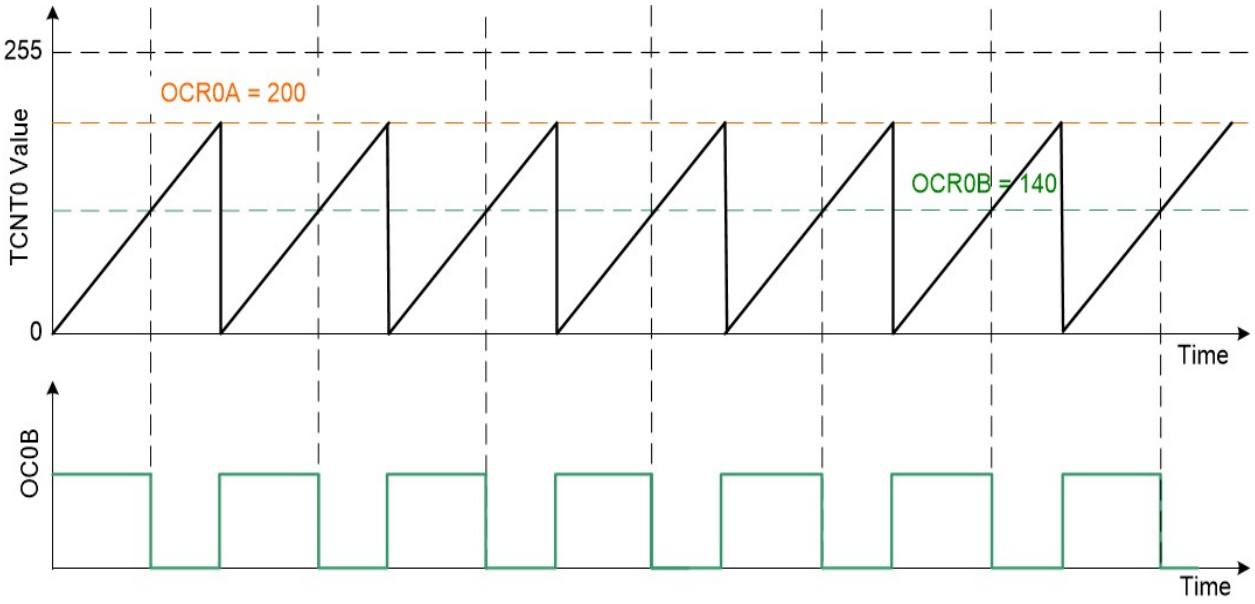
$Mode = 7$, f_{clk_IO} = 8MHz,

$N = 64$, OCR0B = 191, OCR0A = 200,

We Know,

$$f_{OCnx(FPWM)} = \frac{f_{clk\_IO}}{N \times (1 + OCROA)}$$

$$f_{OCR0B(FPWM)} = \frac{8 \times 10^6}{64 \times (1 + 200)} = 621.89Hz$$



How the Waveform is Generated:

- The timer starts from 0, counts up to 200.
- Each time TCNT0 equals OCR0B (140), the **OC0B pin is cleared (goes LOW)**.
- When TCNT0 equals 200 its back to 0, **OC0B is set (goes HIGH)** again.
- This results in a PWM waveform with:
    - **High time** from 0 to 140 counts
    - **Low time** from 140 to 200 counts

Programming Arduino for Phase correct PWM

For Mode 1: Phase correct PWM (TOP = 0xFF = 255):

```
#ifndef F_CPU
#define F_CPU 8000000UL      // Clock frequency is 8 MHz
#endif
#include <avr/io.h>

int main() {
// OC0A pin is set as PWM output pin by sending a HIGH to Port D's Data Direction Register, DDRD
DDRD |= (1<<PD6);
OCR0A = 63; // Load 63 into OCR0A for setting its duty cycle to 75% (See previous example, example 8*)
// Configure TCCR0A and TCCR0B register for (i) inverting (11), (ii) Phase correct PWM mode 1 (001),
// (iii) No Pre-scalar (001) for frequency control. By default, 0s are there, but you may set them explicitly.
TCCR0A |= (1 << COM0A1) | (1 << COM0A0) | (1<<WGM00);
TCCR0B |= (1<<CS00); // CS02:CS00 = 001, N = 1
// TCCR0A=0b11000001; TCCR0B=0b00000001
while(1);
return 0;
}
```

Phase correct PWM Generated Waveform

⇨ **If,**

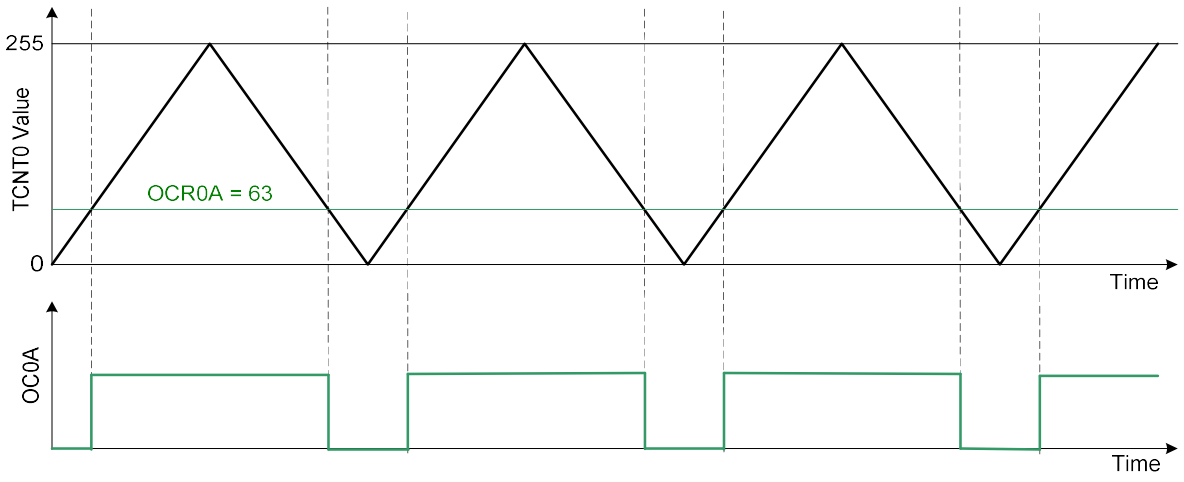
$Mode = 1$, $f_{clk\_IO} = 8\text{MHz}$,

$N = 1$, OCR0A = 63

⇨ **We Know,**

$$f_{OCnx(PCPWM)} = \frac{f_{clk\_IO}}{N \times 510}$$

$$f_{OCR0B(PCPWM)} = \frac{8 \times 10^6}{1 \times 510} = 15.68KHz$$



How the Waveform is Generated:

- The timer starts from **0**, counts **up to 255**, then **back down to 0** (Phase Correct PWM → dual slope).
- Each time TCNT0 equals OCR0A (63), the **OC0A pin is cleared (goes HIGH)**.
- When TCNT0 back to 63, **OC0A is set (goes LOW)** again.
- This results in a PWM waveform with:
    - **LOW time** from 0 to 63 counts
    - **HIGH time** from 63 to 255 counts & then **back down** 255 to 63 counts

Programming Arduino for Phase correct PWM

For Mode 5: Phase correct PWM (TOP = OCR0A):

```
#ifndef F_CPU
#define F_CPU 16000000UL   // Clock frequency is 16 MHz
#endif
#include <avr/io.h>

int main() {
// OC0B pin is set as PWM output pin by sending a HIGH to Port D's Data Direction Register, DDRD
DDRD |= (1<<PD5);
OCR0A = 200; // Load 200 into OCR0A as the TOP value in Phase-Correct PWM mode 5
OCR0B = 63; // Load 63 into OCR0B for setting its duty cycle to 68.5% (See previous example, example 8*)
// Configure TCCR0A and TCCR0B register for (i) inverting (11), (ii) Phase correct PWM mode 5 (101),
// (iii) Pre-scalar (100) for frequency control.
TCCR0A |= (1 << COM0B1) | (1 << COM0B0) | (1<<WGM00);
TCCR0B |= (1<<WGM02) | (1<<CS02); // CS02:00 = 100, N = 256
// TCCR0A=0b00110001; TCCR0B=0b00001100
while(1);
return 0;
}
```

Phase correct PWM Generated Waveform

For Mode 7: Fast PWM (TOP = OCR0A):

⇨ **If,**

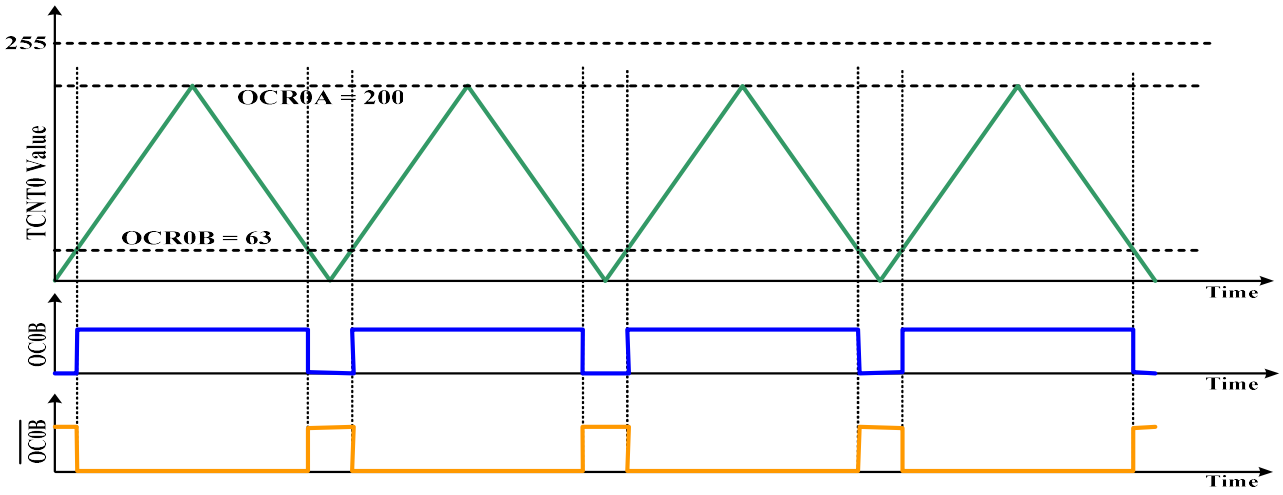
$Mode = 5$, f_{clk_IO} = 16MHz,

$N = 256$, OCR0B = 63, OCR0A = 200,

We Know,

$$f_{OC0x(PCPWM)} = \frac{f_{clk_{IO}}}{2N \times (OCR0A)}$$

$$f_{OCR0B(PCPWM)} = \frac{16 \times 10^6}{2 \times 256 \times 200} = 156.25Hz$$



How the Waveform is Generated:

- The timer starts from **0**, counts **up to 200**, then **back down to 0** (Phase Correct PWM → dual slope).
- Each time TCNT0 equals OCR0A (63), the **OC0B pin is cleared (goes HIGH)**.
- When TCNT0 back to 63, **OC0B is set (goes LOW)** again.
- This results in a PWM waveform with:
    - **LOW time** from 0 to 63 counts
    - **HIGH time** from 63 to 200 counts & then **back down** 200 to 63 counts

Problem1:

(a) **Determine the necessary register setup** to operate a microcontroller in the **fast PWM mode** in **inverting mode**. The counter should count to a **maximum value of 175** and then reset to the BOTTOM and repeat. **Draw the necessary timing waveform**. Use a Timer0 of the Arduino Microcontroller.

(b) **Compute** the **duty cycle** and **sketch the waveform** obtained at port D of the Arduino Microcontroller. **Identify the Timer's modes** of operation and **compute the operating frequency** of that mode based on the following program segment. The system clock frequency is 16 MHz.

```
DDRD |= (1<<PD5);
OCR0A = 200;
OCR0B = 151; // Load OCR0B for setting its duty cycle
// Configure TCCR0A and TCCR0B registers for the scaler
pinMode (5, OUTPUT);
TCCR0A |=  (1 << COM0B1) | (1<<WGM00);
TCCR0B |=  (1<<WGM02) | (1<<CS02) | (1<<CS00)
```

Table 3: Clock select function bits and corresponding pre-scaler values (L) and Compare output mode setting bits (R)

CSn2	CSn1	CSn0	Pre-scaler	COMnA1	COMnA0	Description
0	0	1	1	0	0	The normal port operation, OC0A disconnected
0	1	0	8	0	1	WGM02 = 0; Normal port operation, OC0A disconnected WGM02=1; Toggle OC0A on Compare Match
0	1	1	64	1	0	Clear OC0A on Compare Match, Set OC0A at BOTTOM (non-inverting mode)
1	0	0	256	1	1	Set OC0A on Compare Match, Clear OC0A at BOTTOM (inverting mode)
1	0	1	1024			

(a)

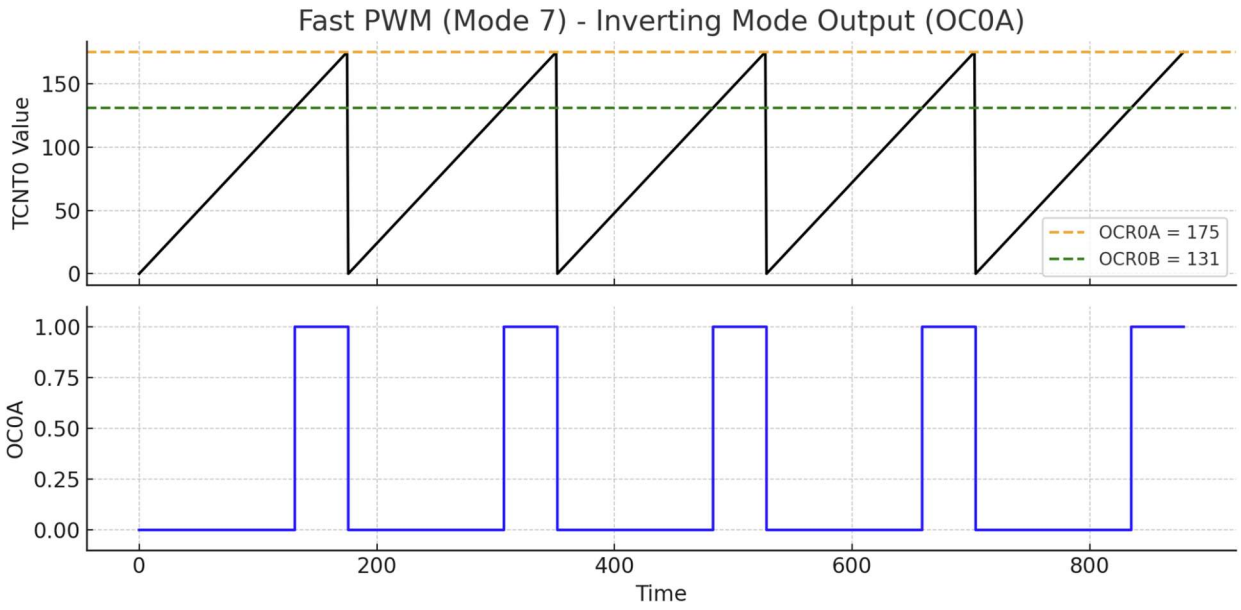
Register Setup:

⇨ We use **Fast PWM Mode 7 (WGM02:0 = 111)** with **OCR0A = 175** (TOP value).
Inverting Mode on OC0A requires **COM0A1 = 1, COM0A0 = 1.**

Register Setup Fast PWM Mode 7:

```
DDRD |= (1 << PD6);      // Set PD6 (OC0A) as output
OCR0A = 175;             // Set TOP value
OCR0B = 131;            // Set duty cycle (can be ignored if OC0B not used)

TCCR0A = (1 << COM0A1) | (1 << COM0A0) | (1 << WGM01) | (1 << WGM00); // Inverting mode
TCCR0B = (1 << WGM02) | (1 << CS00);  // Fast PWM Mode 7 with N = 1
```



(b)

⇨ **Is given,**

f_{clk_IO} = 16MHz, OCR0B = 151, OCR0A = 200,

$N = 1024$, *since* $Cs02 = 1, Cs01 = 0, Cs00 = 1$

Identify the Timer's modes

Mode = non-inverting mode *since* $COM0B1 = 1, COM0B0 = 0$

PWM Mode = 5 *since* $WGM02 = 1, WGM01 = 0, WGM02 = 1$ (∴ Phase correct PWM Mode)

We Know,

For operating frequency:

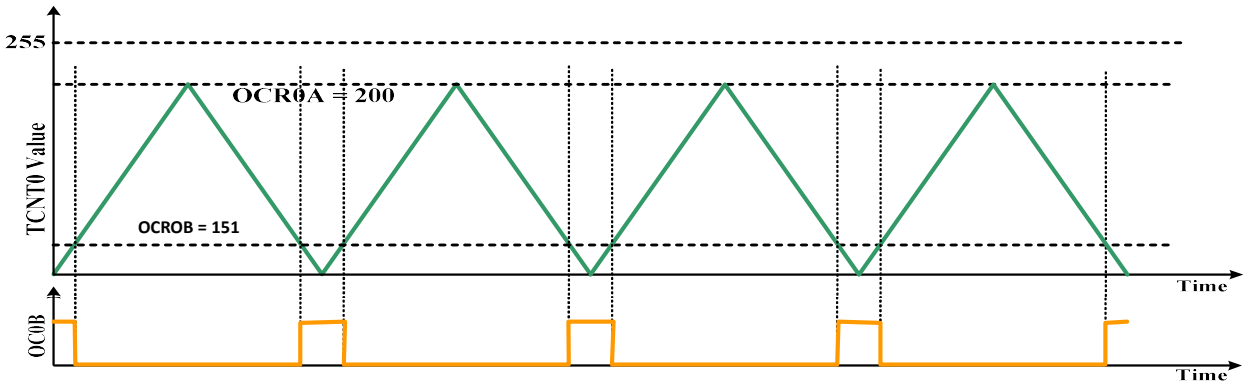
$$f_{OC0x(PCPWM)} = \frac{f_{clk_{IO}}}{2N \times (OCR0A)}$$

$$f_{OCR0B(PCPWM)} = \frac{16 \times 10^6}{2 \times 1024 \times 200} = 39.06Hz$$

For Duty cycle:

$$\mathbf{OCR0B} = \left(\frac{(\mathbf{OCR0A}) \times \boldsymbol{D}}{\mathbf{100}}\right)$$

$$\mathbf{D} = \left(\frac{\mathbf{OCR0B}}{\mathbf{OCR0A}} \times \mathbf{100}\right) = \frac{\mathbf{151}}{\mathbf{200}} \times \mathbf{100} = \mathbf{75.5\%}$$



PROCESSOR LOGIC DESIGN
(1ST PART)
ALU DESIGN

Introduction

- **Processor Unit**: Executes operations in a digital system.
- **Components**: Includes registers and circuits for arithmetic, logic, shift, and data transfer.
- **CPU**: Processor Unit + Control Unit (CU).
- **CPU Parts**: Register Banks (Memory), ALU, and CU.
- **Processor Organizations**:
    - Bus Organization
    - Scratchpad Memory
    - Two-Port Memory

Bus Organization

The **main control signals** required to perform the micro-operation `R1 = R2 + R3` in a bus organization with 4 registers and 2 buses (A and B):

Control Signals and Their Functions:

⇨ `Let,` registers are coded as R0=00, R1=01, R2=10, R3=11

1. **MUX A Selector**
    - **Purpose**: Load `R2` onto **Bus A**
    - **Selection Code**: Binary code for `R2` (e.g., `10`)
2. **MUX B Selector**
    - **Purpose**: Load `R3` onto **Bus B**
    - **Selection Code**: Binary code for `R3` (e.g., `11`)
3. **ALU Function Selector**
    - **Purpose**: Perform **A + B** operation
    - **Selection Code**: Code for **addition** in ALU (depends on ALU design)
4. **Shift Selector**
    - **Purpose**: **No shift** (direct transfer from ALU to output bus `S`)
    - **Selection Code**: Code for "No shift" (e.g., `00`)
5. **Decoder Destination Selector**
    - **Purpose**: Store result from **Bus S** into **R1**
    - **Selection Code**: Binary code for `R1` (e.g., `01`)

Summary Table:

Control Unit Selector	Target	Function	Example Code
MUX A Selector	Bus A	Load R2	`10`
MUX B Selector	Bus B	Load R3	`11`
ALU Function Selector	ALU	Perform A + B	`ADD` code
Shift Selector	Output Bus S	No shift	`00`
Decoder Destination Selector	Register	Store result in R1	`01`

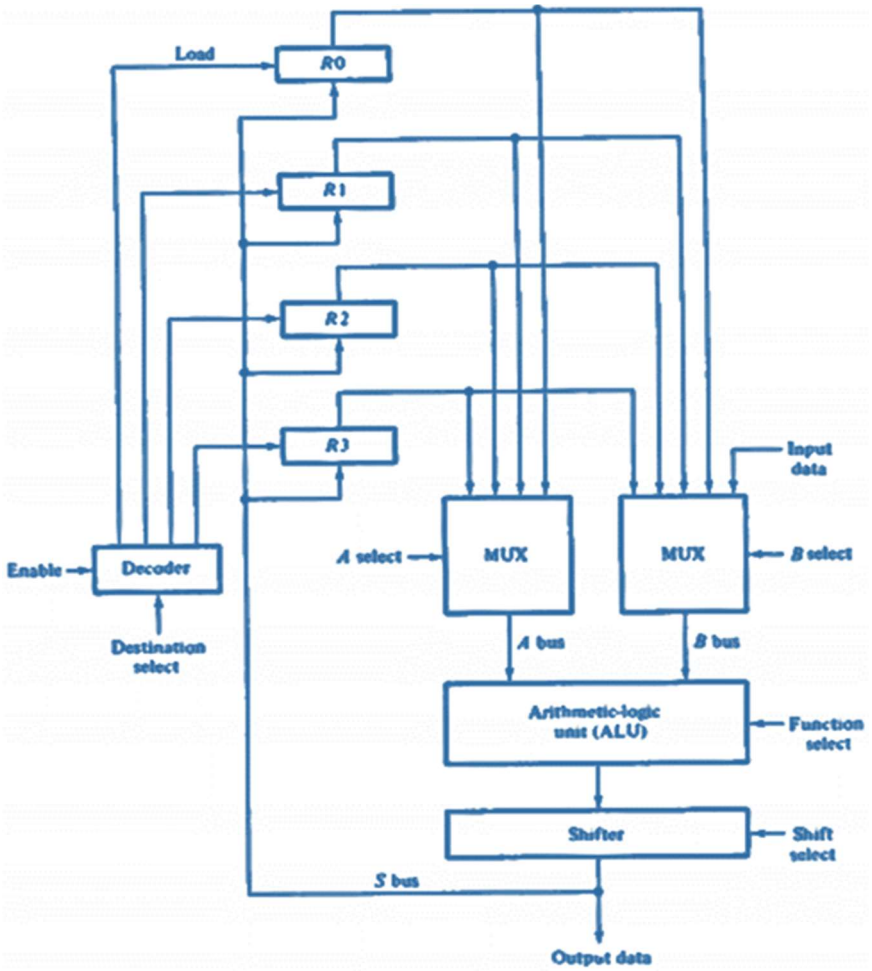
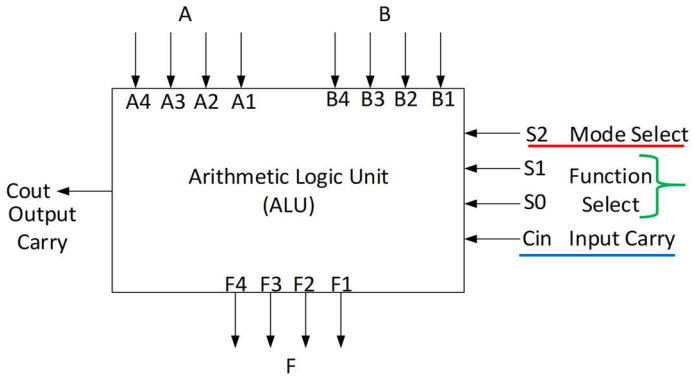


Fig. 9.1 Processor registers and ALU connected through Common Buses

- **All 5 control signals** must be generated **simultaneously** within **one clock pulse**.
- **Data flows** from source registers → MUXes → ALU → Shifter → Output Bus → Destination register **within one clock cycle**.
- On the **next clock pulse**, result is **loaded into R1**.

Arithmetic Logic Unit (ALU)

- **ALU**: A **combinational circuit** for arithmetic and logic operations.
- Performs **basic arithmetic & logic functions**.
- Uses **selection lines** to choose operations.
- **k selection lines** can select 2^K **operations** via internal decoding.



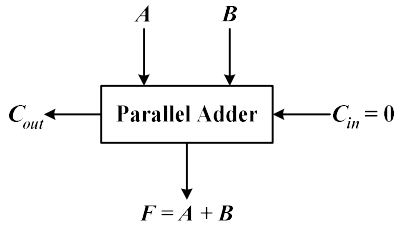
1. **s2 (Mode Select)**: Chooses between **arithmetic** and **logic** operations.
2. **s1 & s0 (Function Select)**: Specify the exact operation.
3. **Cin**: Used only in **arithmetic operations**.
4. **Cin** often acts as a **4th selection variable**, enabling **8 arithmetic** and **4 logic operations**.

Design of Arithmetic Circuit

1. **Parallel Adder** is the core of the ALU's arithmetic section.
2. Built using **multiple cascaded full adders**.
3. **Different arithmetic operations** are achieved by controlling its **data inputs**.

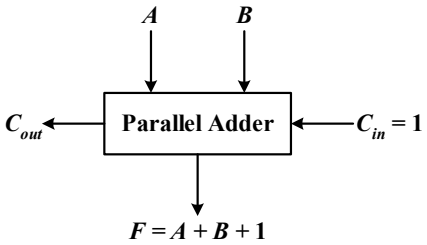
Operations obtained by controlling one set of inputs to a parallel adder

S1	S0	C_{in}
0	1	0



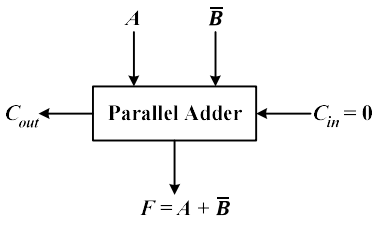
(a) Addition without input carry

S1	S0	C_{in}
0	1	1



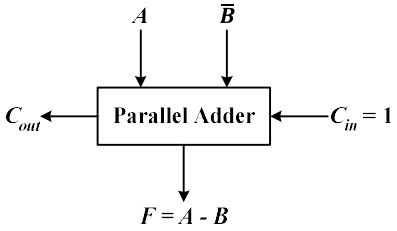
(b) Addition with input carry

S1	S0	C_{in}
1	0	0



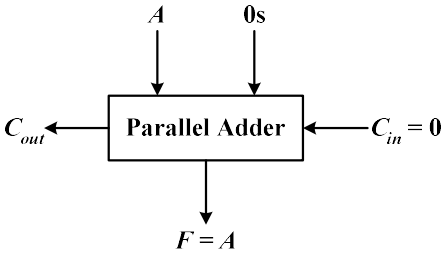
(c) Addition with 1's complement of B

S1	S0	C_{in}
1	0	1



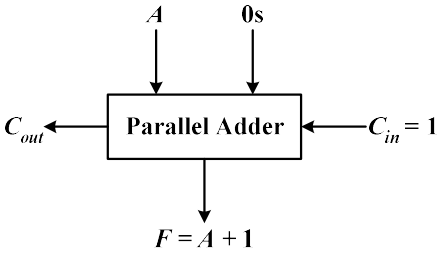
(d) Subtraction

S1	S0	C_{in}
0	0	0



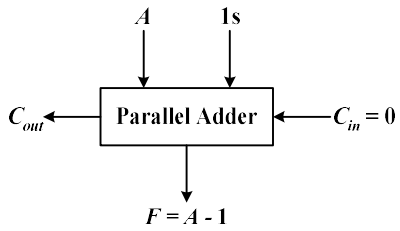
(e) Transfer of A

S1	S0	C_{in}
0	0	1



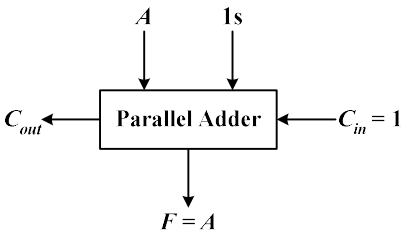
(f) Increment of A

S1	S0	C_{in}
1	1	0



(g) Decrement of A

S1	S0	C_{in}
1	1	1

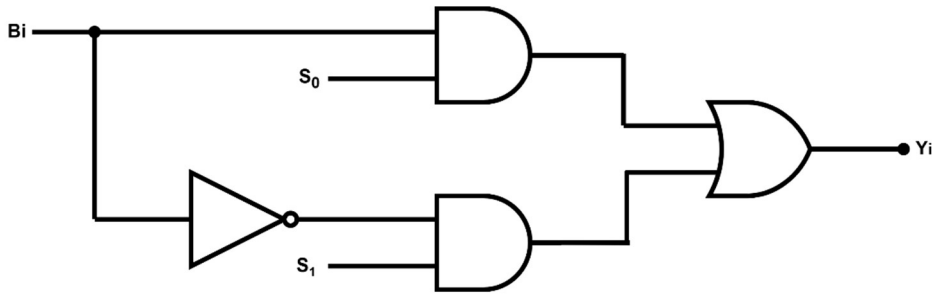


(h) Transfer of A

Function Table for Arithmetic Circuit

Function selects			X equals	Y equals	Output equals	Function
S_1	S_0	C_{in}				
0	0	0	A	0	F = A	Transfer A
0	0	1	A	0	F = A + 1	Increment A
0	1	0	A	B	F = A + B	Add B to A
0	1	1	A	B	F = A + B + 1	Add B to A plus 1
1	0	0	A	$\overline{\mathbf{B}}$	F = A + $\overline{\mathbf{B}}$	Add 1's complement of B to A
1	0	1	A	$\overline{\mathbf{B}}$	F = A + $\overline{\mathbf{B}}$ + 1	Add 2's complement of B to A
1	1	0	A	All 1's	F = A – 1	Decrement A
1	1	1	A	All 1's	F = A	Transfer A

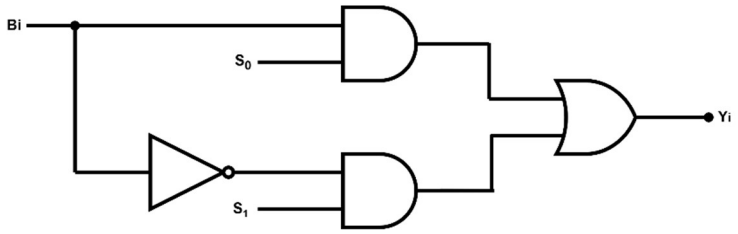
Functions of Arithmetic Circuit



s1	s0	Yi (Output)
0	**0**	0
0	**1**	Bi
1	**0**	$\overline{\text{Bi}}$
1	**1**	1

- The circuit is a **true/complement, one/zero element** that controls **input B**.
- **Controlled by two selection lines**: **`s1` and `s0`.**
- There are **n identical circuits** (for each Bi).
- **Output behavior (Yi):**
- Uses **AND and OR gates** to generate the desired logic output.

True/Complement, One/Zero Circuit



S_1	S_0	Yi (Output)
0	**0**	0
0	**1**	Bi
1	**0**	$\overline{Bi}$
1	**1**	1

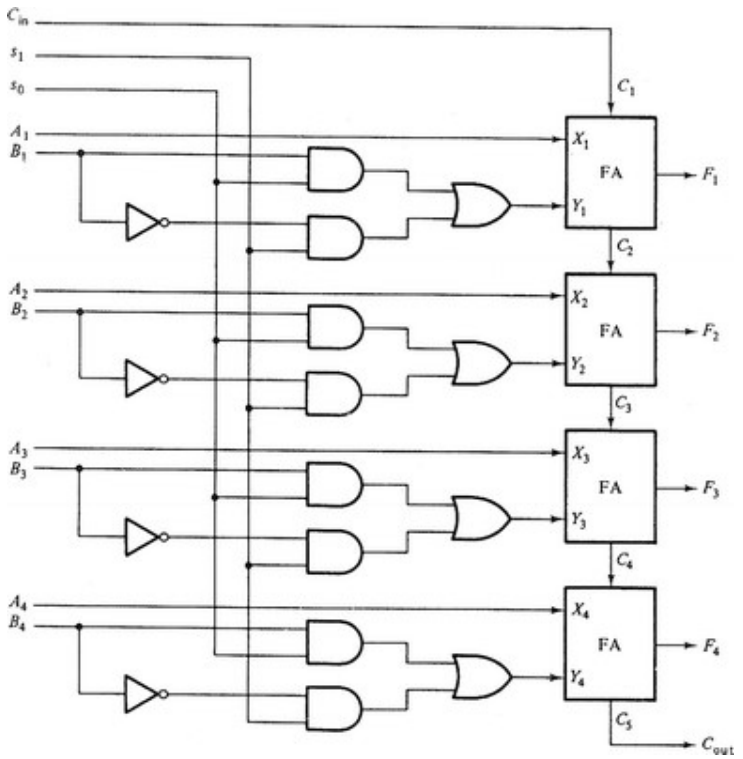
» $Yi = (Bi \cdot S_0) + (\overline{Bi} \cdot S_1)$ **for i = 1, 2, ..., n**

Logic Diagram of a 4-Bit Arithmetic Circuit

- **Combinational logic** per stage is defined by:

» $Xi = Ai$

» $Yi = (Bi \cdot S_0) + (\overline{Bi} \cdot S_1)$



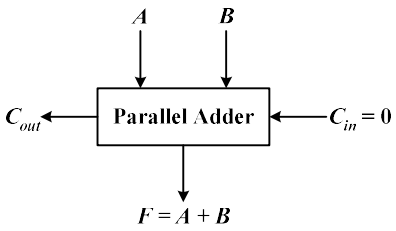
Problem1: **Design a 4-bit adder/subtractor circuit** with **one selection variable S** and **two inputs A & B**: when **S = 0 the circuit performs A+B**. When **S = 1 the circuit performs A-B** by taking the **2's complement of B.**

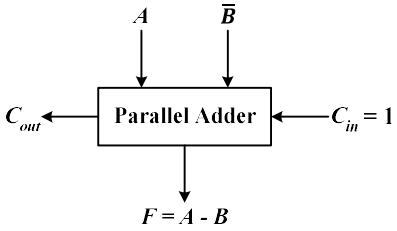
⇨ **is given,**

S	X	Y	F	C
0	A	B	$A + B$	0
1	A	$\overline{B}$	$A - B$	1

⇨ **We Know,**

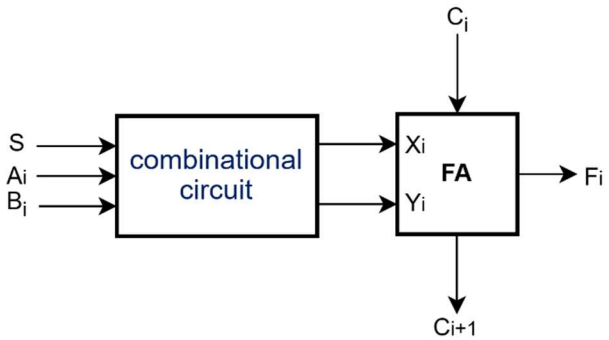
Function Specification:





Specifying the combinational circuit:

S	X_i	Y_i	C_{in}
0	A_i	B_i	0
1	A_i	$\overline{B}_i$	1



≫Truth Table for specified equations:

S	A_i	B_i	X_i	Y_i
0	0	0	0	0
0	0	1	0	1
0	1	0	1	0
0	1	1	1	1
1	0	0	0	1
1	0	1	0	0
1	1	0	1	1
1	1	1	1	0

≫Equations of the outputs:

$$\gg Xi = Ai$$

$$\gg Yi = Bi \oplus S$$

Truth Table of XOR gate

INPUT		OUTPUT
A	B	C
0	0	0
0	1	1
1	0	1
1	1	0

∴4-bit adder/subtractor circuit:

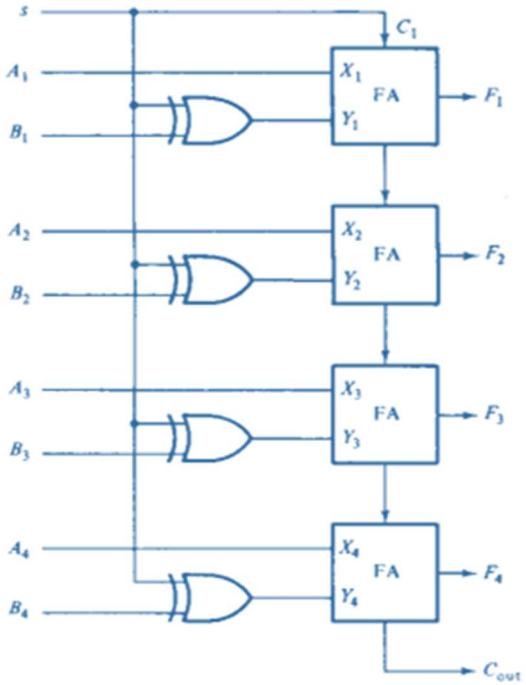
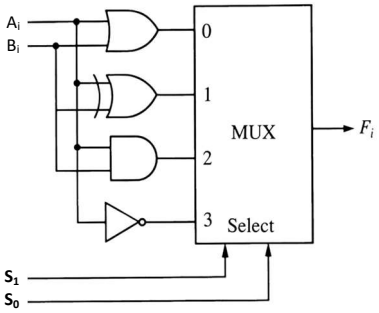


Figure 9-10 4-bit adder/subtractor circuit

Design of a Logic Circuit

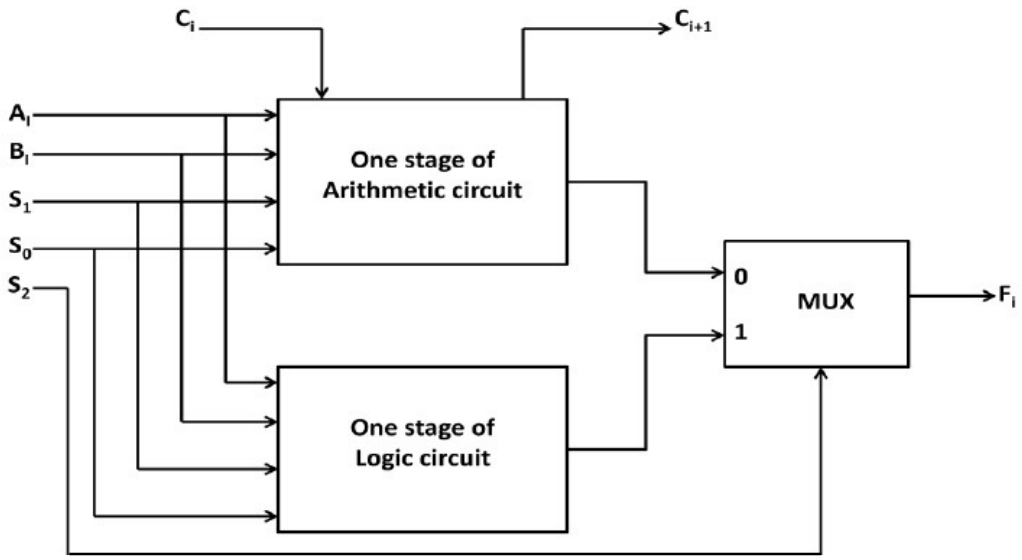
- **Logic microoperations** treat each bit **independently** as a binary variable.
- All logic functions are built using **AND, OR, NOT**.
- **2 selection lines** can select **4 operations** → allows adding **XOR** as the 4th operation.
- Thus, logic circuit performs **AND, OR, NOT, XOR**.



S_1	S_0	Output	Operation
0	0	$F_i = A_i + Bi$	OR
0	1	$F_i = A_i \oplus Bi$	XOR
1	0	$F_i = A_i Bi$	AND
1	1	$F_i = \bar{A}_i$	NOT

Combining Logic and Arithmetic Circuits

- **Logic and arithmetic circuits** can be **combined into one ALU**.
- Use **common selection lines**: **`s1` and `s0`.**
- A **third variable `s2`** differentiates between logic and arithmetic operations.
- **s2** acts as the **select input** to a **2:1 multiplexer** that chooses the final output.



Logic Operations in One Stage of Arithmetic Circuit

S_2	S_1	S_0	X_i	Y_i	C_i	$F_i = X_i \oplus Y_i$	Operation	Required Operation
1	0	0	A_i	0	0	$F_i = A_i$	Transfer A	OR
1	0	1	A_i	B_i	0	$F_i = A_i \oplus B_i$	XOR	XOR
1	1	0	A_i	$\overline{B_i}$	0	$F_i = A_i \odot B_i$	Equivalence	AND
1	1	1	A_i	1	0	$F_i = \overline{A_i}$	NOT	NOT

Table. 9.1 Logic operations in one state of arithmetic circuit

➔ By modifying A_i, we can achieve the required operation.

S_2	S_1	S_0	X_i	Y_i	C_i	$F_i = X_i \oplus Y_i$	Operation	Required Operation
1	0	0	A_i+B_i	0	0	$F_i = A_i + B_i$	OR	OR
1	0	1	A_i	B_i	0	$F_i = A_i \oplus B_i$	XOR	XOR
1	1	0	$A_i+\overline{B_i}$	$\overline{B_i}$	0	$F_i = A_i B_i$	AND	AND
1	1	1	A_i	1	0	$F_i = \overline{A_i}$	NOT	NOT

Selection Control:

- Controlled by **S2 = 1** (to indicate logic operation mode)
- **S1** and **S0** select the specific logic function
- **Carries are ignored in logic operations**

Note: In a full ALU, logic operations are merged with arithmetic operations using a **multiplexer**, and `S2` determines which output is selected (arithmetic or logic).

Structure of the ALU Using Full-Adders

⇨ **Each full adder gets three inputs:**

- `Xi` = **input 1**
- `Yi` = **input 2**
- `Ci` = **carry-in**

⇨ The **output** is:

- `Fi = Xi ⊕ Yi ⊕ Ci` **for arithmetic mode**
- `Fi = Xi ⊕ Yi` **for logic mode (since** `Ci = 0` **when** `s2 = 1`**)**

To produce **logic operations**, we cleverly modify `Xi` and `Yi` using Boolean expressions based on select inputs `s2s1s0`.

⇨ **Main Idea:**

- Use selection inputs **s2, s1, s0** to choose between **arithmetic** and **logic** functions.
- `s2 = 0` → Arithmetic operations.
- `s2 = 1` → Logic operations (use `Ci = 0` and modify inputs).

⇨ **Boolean Functions for ALU Inputs:**

- **For Arithmetic (s2 = 0):**
  - **Xi = Ai**
  - **Yi = s0 ·Bi + s1 ·$\overline{Bi}$**
  - **Zi = Ci**
- **For Logic (s2 = 1):**
  - **Xi = Ai + $\overline{s1}$ ·$\overline{s0}$ ·Bi + s1 ·$\overline{s0}$ ·$\overline{Bi}$**
  - **Yi = s0 ·Bi + s1 ·$\overline{Bi}$**
  - **Zi = 0**

Design of Arithmetic Logic Circuit

⇨ **Design a 2-bit ALU for the operations listed below in the Table:**

Binary Code	B	A	F with $C_{in} = 0$	F with $C_{in} = 1$	D	H
0 0 0	Input Data	Input Data	A	A+1	None	Circulate-Left with Carry
0 0 1	R1	R1	A + B	A + B + 1	R1	Circulate-Right with Carry
0 1 0	R2	R2	A + B′	A + B′ + 1	R2	No shift
0 1 1	R3	R3	A − 1	A	R3	0's to the output Bus
1 0 0	R4	R4	A OR B	A OR B	R4	–
1 0 1	R5	R5	A XOR B	A XOR B	R5	Shift Left with $I_L = 0$
1 1 0	R6	R6	A AND B	A AND B	R6	Shift Right with $I_R = 0$
1 1 1	R7	R7	A′	A′	R7	1's to the output Bus

⇨ **Let's construct the truth table based on the table provided above:**

S_2	S_1	S_0	C_{in}	X_i	Y_i	Z_i
0	0	0	0	A	0	0
0	0	0	1	A	0	1
0	0	1	0	A	B	0
0	0	1	1	A	B	1
0	1	0	0	A	$\overline{B}$	0
0	1	0	1	A	$\overline{B}$	1
0	1	1	0	A	1	0
0	1	1	1	A	1	1
1	0	0	X	A + B	0	0
1	0	1	X	A	B	0
1	1	0	X	A + $\overline{B}$	$\overline{B}$	0
1	1	1	X	A	1	0

⇨ **Now draw the K-Map for output equations:**

➢ **For Xi:**

S_2S_1 \ S_0	0	1
00	A	A
01	A	A
11	A+$\overline{B}$	A
10	A+B	A

$$X_i = A_i + S_2S_1\overline{S}_0\overline{B}_i + S_2\overline{S}_1\overline{S}_0B_i$$

➢ **For Yi:**

S_2S_1 \ S_0	0	1
00	0	B
01	$\overline{B}$	1
11	$\overline{B}$	1
10	0	B

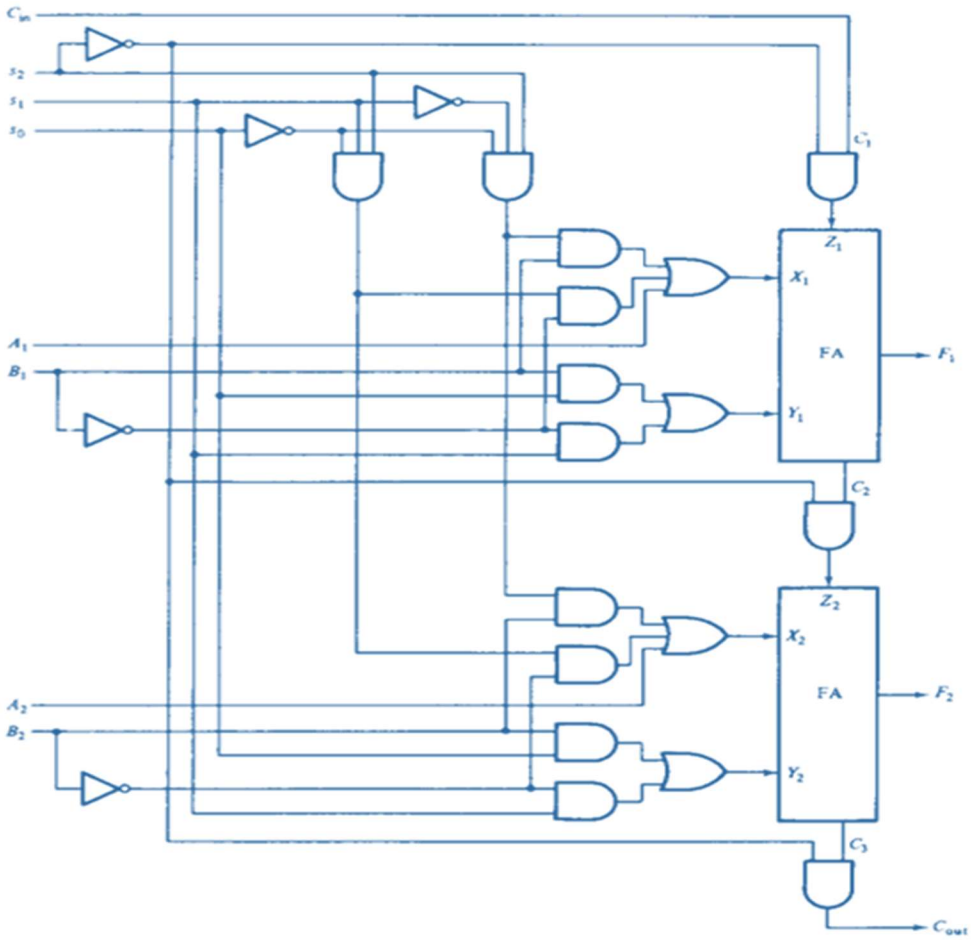
$$Y_i = S_1\overline{B}_i + S_0B_i$$

➢ **For Zi:**

S_2S_1 \ S_0C_{in}	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	0	0	0	0
10	0	0	0	0

$$Z_i = \overline{S}_2C_{in}$$

∴ **2-bit ALU circuit:**



Problem2: Sketch a 2-bit Arithmetic Logic Unit (ALU) for the operations listed in Table 1. Show the (A+ B) operation using bus organized processor for $A = R1 = 101$ *and* $B = R3 = 110. R0 = A + B.$

Binary Code	Function of selection variables					
	A	B	D	F with $C_{in} = 0$	F with $C_{in} = 1$	H
0 0 0	Input Data	Input Data	None	$A - 1$	$A, C \leftarrow 1$	No Shift
0 0 1	R1	R1	R1	$A + B$	$A + B + 1$	Shift Right with $I_R = 0$
0 1 0	R2	R2	R2	$A - B - 1$	$A - B$	Shift Left with $I_L = 0$
0 1 1	R3	R3	R3	$A, C \leftarrow 0$	$A + 1$	Circulate Left with Carry
1 0 0	R4	R4	R4	A'	A'	0's to the output Bus
1 0 1	R5	R5	R5	$A \cap B$	$A \cap B$	1's to the output Bus
1 1 0	R6	R6	R6	A XOR B	A XOR B	Circulate Right with Carry
1 1 1	R7	R7	R7	$A \cup B$	$A \cup B$	-

⇨ **Let's construct the truth table based on the table provided above:**

S_2	S_1	S_0	C_{in}	X_i	Y_i	Z_i
0	0	0	0	A	1	0
0	0	0	1	A	1	1
0	0	1	0	A	B	0
0	0	1	1	A	B	1
0	1	0	0	A	$\overline{B}$	0
0	1	0	1	A	$\overline{B}$	1
0	1	1	0	A	0	0
0	1	1	1	A	0	1
1	0	0	X	A	1	0
1	0	1	X	A + $\overline{B}$	$\overline{B}$	0
1	1	0	X	A	B	0
1	1	1	X	A + B	0	0

⇨ **Now draw the K-Map for output equations:**

➢ **For Xi:**

S_2S_1 \ S_0	0	1
00	A	A
01	A	A
11	A	A+B
10	A	A+$\overline{B}$

$$X_i = A_i + S_2\bar{S_1}S_0\bar{B_i} + S_2S_1S_0B_i$$

➢ **For Yi:**

S_2S_1 \ S_0	0	1
00	1	B
01	$\overline{B}$	0
11	B	0
10	1	$\overline{B}$

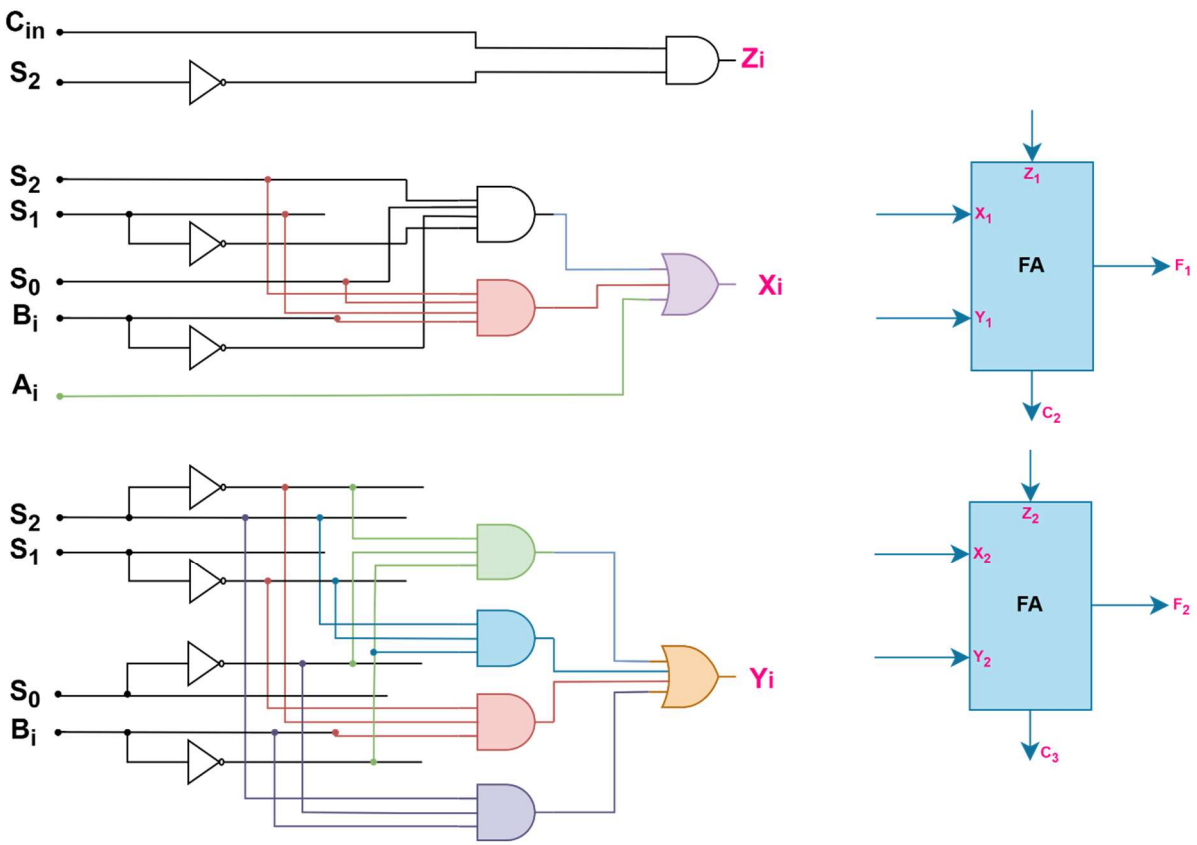
$$Y_i = \bar{S_2}\bar{S_0}\bar{B_i} + S_2\bar{S_1}\bar{B_i} + \bar{S_2}\bar{S_1}B_i + S_2\bar{S_0}B_i$$

➢ **For Zi:**

S_2S_1 \ S_0C_{in}	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	0	0	0	0
10	0	0	0	0

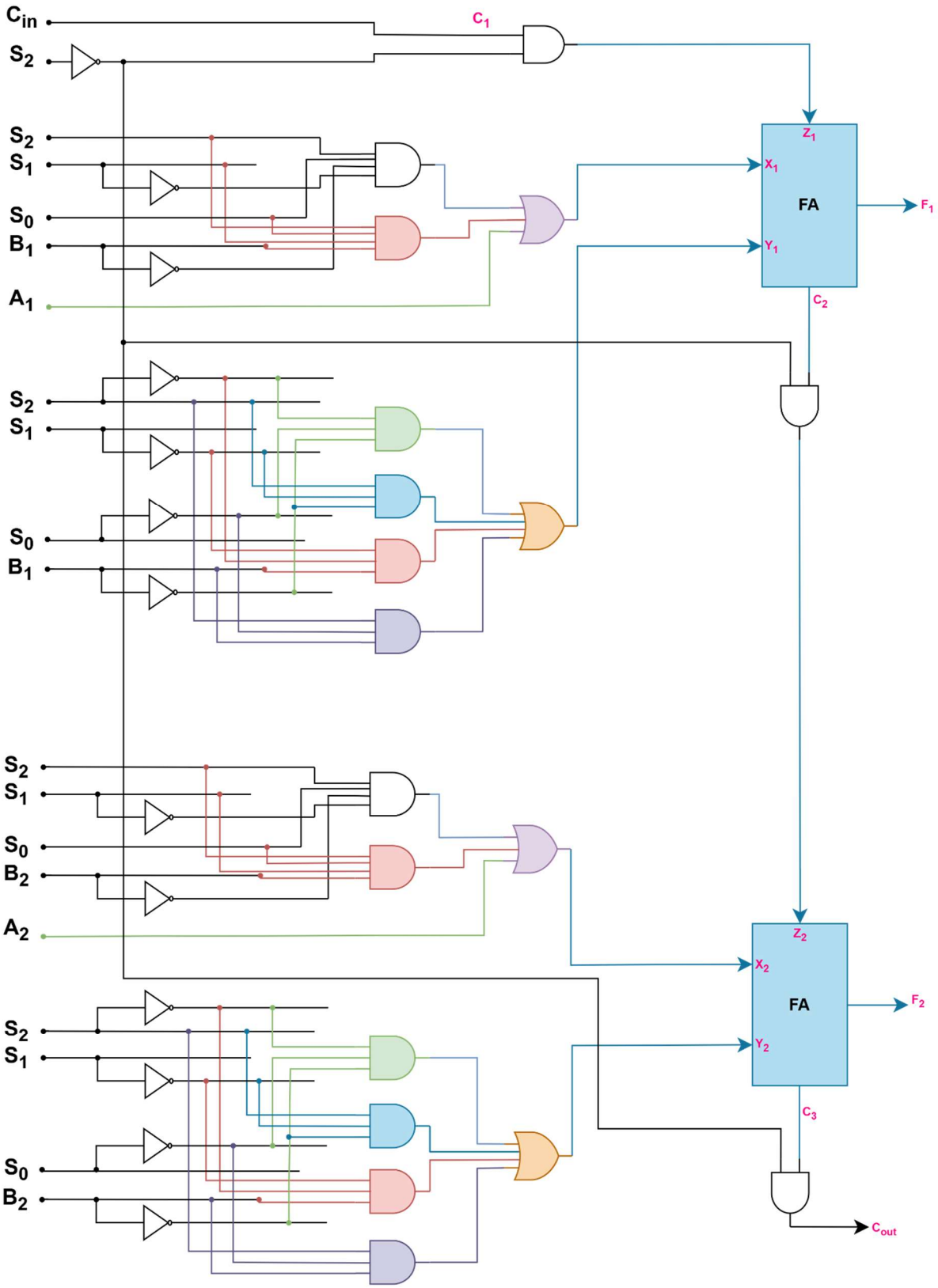
$$Z_i = \bar{S_2}C_{in}$$

∴ 2-bit ALU circuit:



Control Unit Selector	Target	Function	Selection Code	value
MUX A Selector	Bus A	Load R1	001	101
MUX B Selector	Bus B	Load R3	010	110
ALU Function Selector	ALU	Perform A + B	0010	1011
Shift Selector	Output Bus S	No shift	000	1011
Decoder Destination Selector	Register	Store result in R0	000	1011

∴ 2-bit ALU Complete circuit:

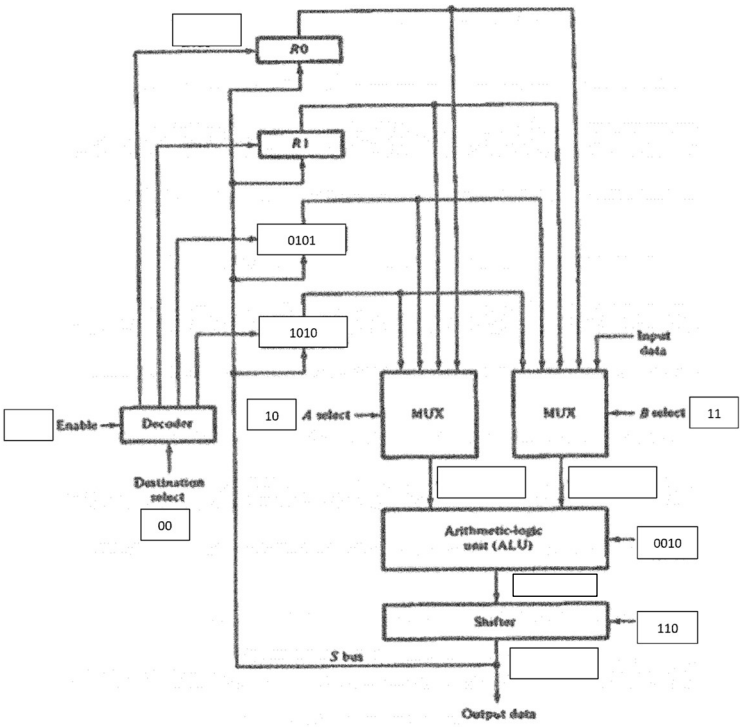


Problem3: Show the operation using the bus organized processor and based on the operations listed in Table 1.

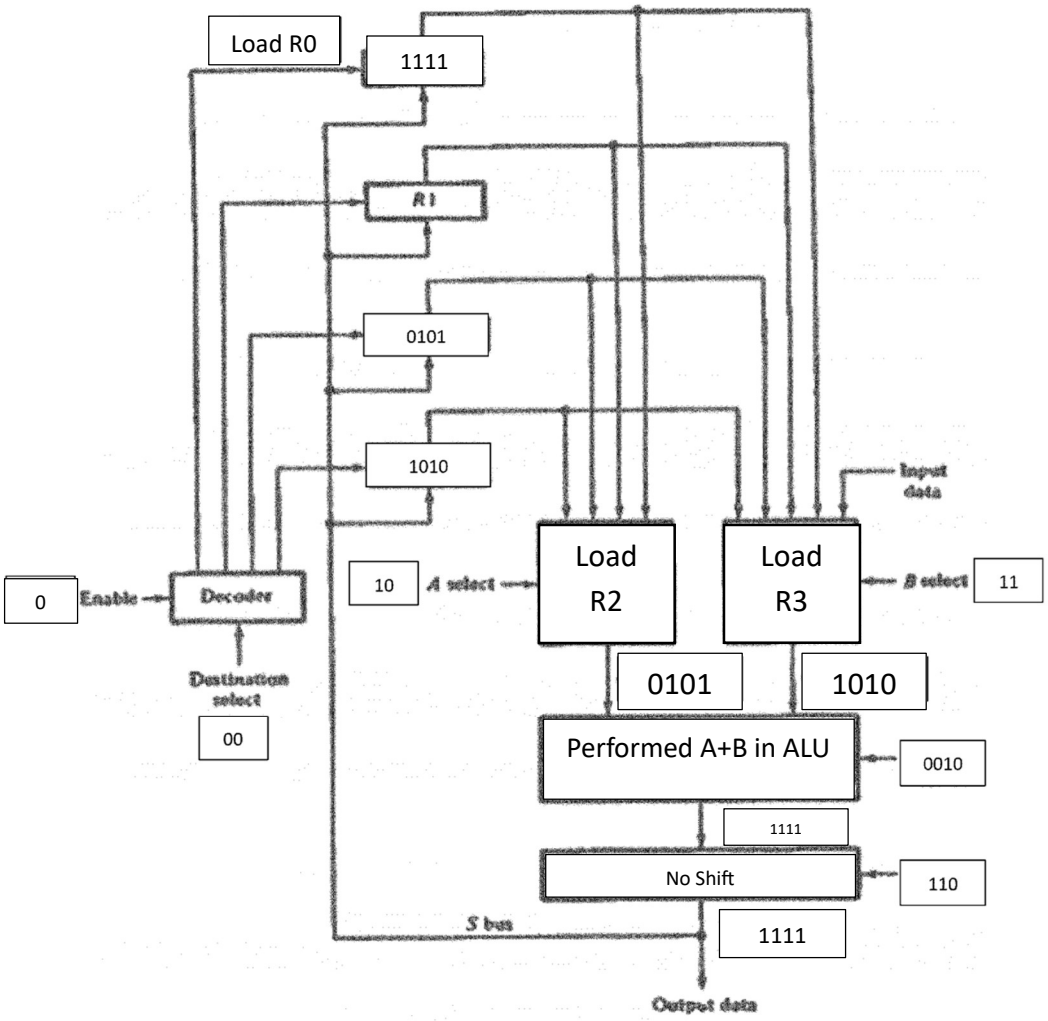
For each binary selection code (A, B, D), explain how the data flows through the system, which registers are selected, what function the ALU performs, and what shifting operation is applied.

Use the structure and components (MUX, ALU, Shifter, Registers) from the provided diagram to explain the operations step-by-step.

Binary Code	Function of selection variables					
	A	B	D	F with $C_{in} = 0$	F with $C_{in} = 1$	H
0 0 0	Input Data	Input Data	None	$A - 1$	$A, C \leftarrow 1$	Circulate Right with Carry
0 0 1	R1	R1	R1	$A + B$	$A + B + 1$	Shift Right with $I_R = 0$
0 1 0	R2	R2	R2	$A - B - 1$	$A - B$	Shift Left with $I_L = 0$
0 1 1	R3	R3	R3	$A, C \leftarrow 0$	$A + 1$	Circulate Left with Carry
1 0 0	R4	R4	R4	A'	A'	0's to the output Bus
1 0 1	R5	R5	R5	$A \cap B$	$A \cap B$	1's to the output Bus
1 1 0	R6	R6	R6	A XOR B	A XOR B	No Shift
1 1 1	R7	R7	R7	$A \cup B$	$A \cup B$	-



⇨ **Let's Use the structure and components (MUX, ALU, Shifter, Registers) from the provided diagram to explain the operations step by step**



Control Unit Selector	Target	Function	Selection Code	value
MUX A Selector	Bus A	Load R2	010	0101
MUX B Selector	Bus B	Load R3	011	1010
ALU Function Selector	ALU	Perform A + B	0010	1111
Shift Selector	Output Bus S	No shift	110	1111
Decoder Destination Selector	Register	Store result in R0	000	1111

PROCESSOR LOGIC DESIGN
(2ND PART)
STATUS REGISTER AND SHIFTER DESIGN

Design of Status Register

➢ Status Bit in ALU

- ALUs often include a **status register**.
- It stores **status bits** (also called **condition-code** or **flag bits**).
- These bits indicate:
  - **Overflow**
  - **Zero result**
  - **Sign (positive/negative)**
- Useful for further analysis and control decisions in processing.

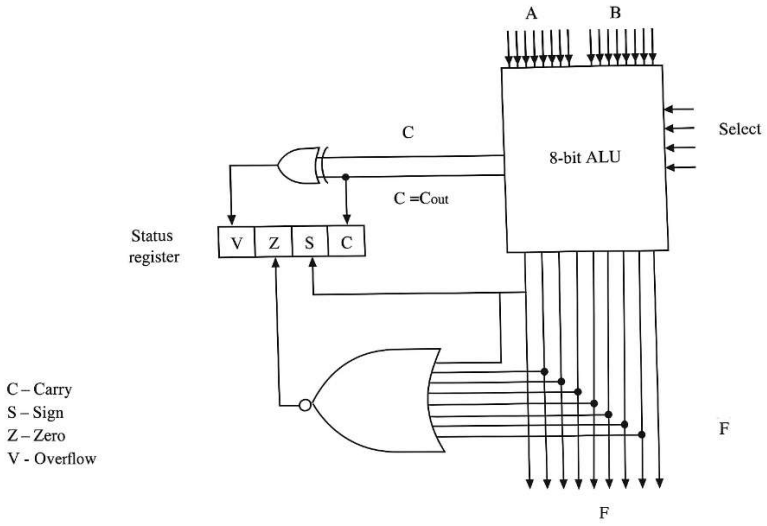


Fig9.14: 8-bit ALU with a 4-bit status register

➢ Status Bits:

- **C (Carry)**:
  - Set if ALU generates a carry-out (1).
  - Cleared if no carry-out (0).
- **S (Sign)**:
  - Set if MSB (bit 7) of result is 1 (negative).
  - Cleared if MSB is 0 (positive).
- **Z (Zero)**:
  - Set if result = 0 (all bits are 0).
  - Cleared otherwise.
- **V (Overflow)**:
  - Set if overflow occurs in 2's complement addition:
    $V = C9 \oplus C8$
  - Happens when result > 127 or < -128.

Status bits after the subtraction operation of two unsigned numbers (*A-B*)

Relation	Condition of status bits	Boolean function
A > B	C = 1 and Z = 0	CZ′
A ≥ B	C = 1	C
A < B	C = 0	C′
A ≤ B	C = 0 or Z = 1	C′ + Z
A = B	Z = 1	Z
A ≠ B	Z = 0	Z′

Status bits after the subtraction (*A-B*) of two signed binary numbers when negative numbers are in 2's complement form

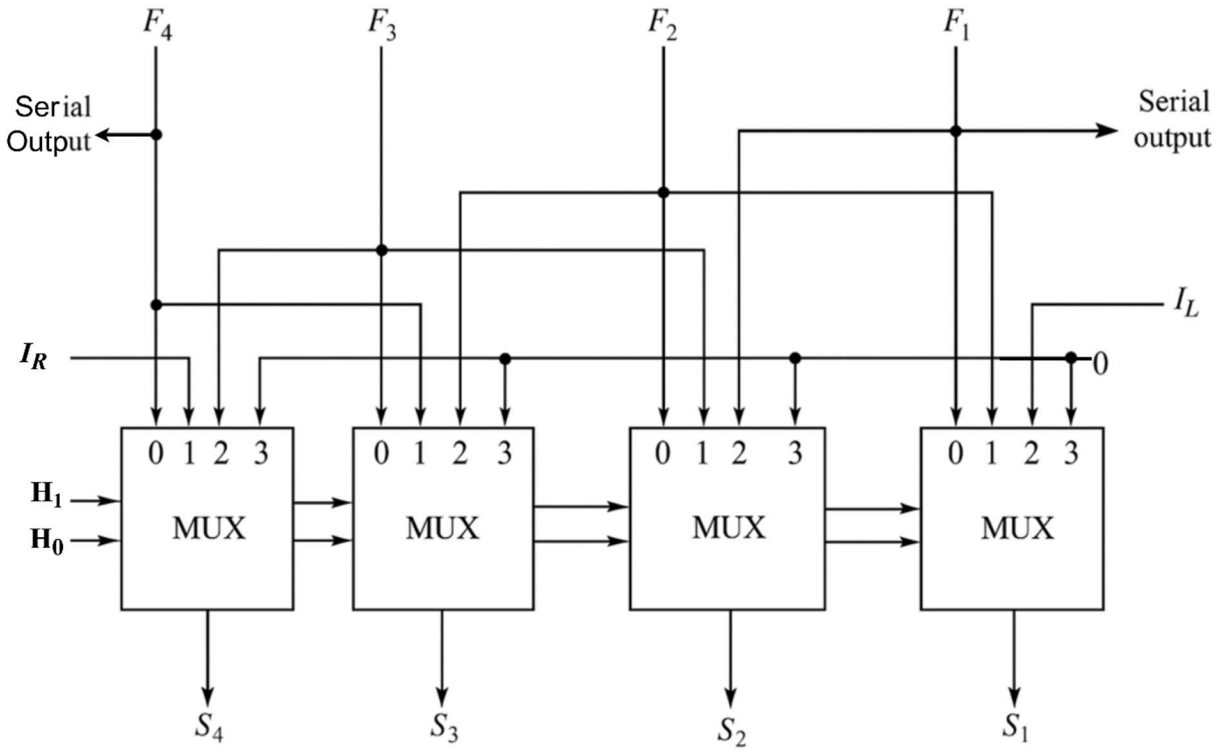
Relation	Condition of status bits	Boolean function
A > B	Z = 0 and (S = 0, V = 0 or S = 1, V = 1)	Z′(S ⊙ V)
A ≥ B	S = 0, V = 0 or S = 1, V = 1	S ⊙ V
A < B	S = 1, V = 0 or S = 0, V = 1	S ⊕ V
A ≤ B	S = 1, V = 0 or S = 0, V = 1 or Z = 1	(S ⊕ V) + Z
A = B	Z = 1	Z
A ≠ B	Z = 0	Z′

➢ C Bit as Borrow (in Subtraction)

- **After `A - B`:**
  - **No borrow** if `A ≥ B`
  - **Borrow needed** if `A < B`
- **Borrow Condition**:
  - Is the **complement** of carry-out from 2's complement subtraction?
- **Processor Behavior**:
  - Some CPUs **invert the C bit** after subtraction.
  - The complemented **C bit is treated as the borrow flag**.

Design of a Shifter

- The shift unit **transfers ALU output** to the output bus.
- It can perform **left shift, right shift, or no shift.**
- Sometimes, **no transfer from ALU** to the output bus is also allowed.
- It handles shift operations not typically supported by the ALU.



H1	H0	Operation	Description
0	**0**	S ← F	**Transfer F** into the output Bus, S (No Shift)
0	**1**	S ← shr F	**Shift Right** F into the output Bus, S
1	**0**	S ← shl F	**Shift Left** F into the output Bus, S
1	**1**	S ← 0	**Transfer 0**'s into the output Bus, S

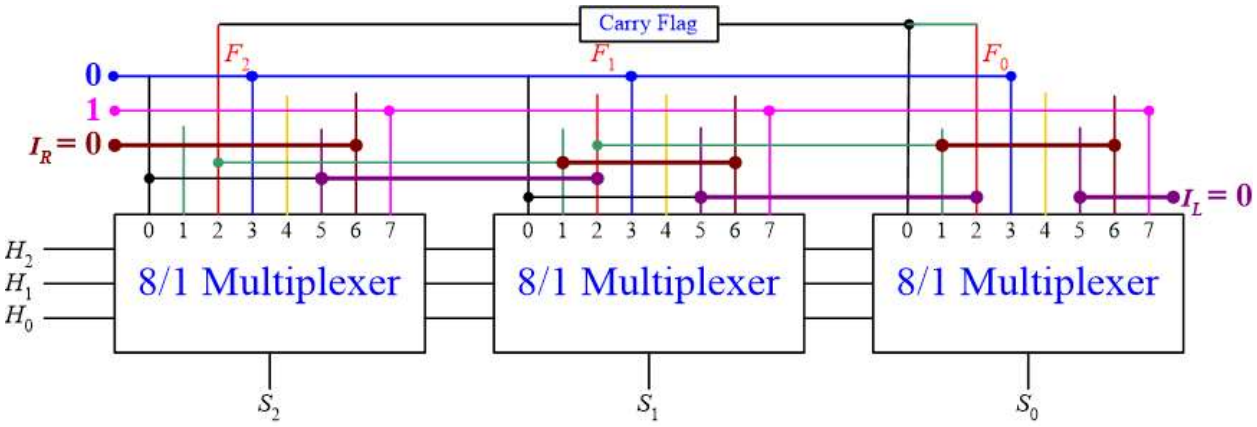
To add more operations:

- Use **8x1 MUX** with **third select line H2**.
- For **Carry-based shifts**:
  - **CLC L** = Circulate Left with Carry (uses $I_L = C$)
  - **CRC R** = Circulate Right with Carry (uses $I_R = C$)

➢ **Design a 3-bit shifter for the shifting operations listed in the following Table:**

Binary Code	Function of selection variables					
	B	A	F with $C_{in} = 0$	F with $C_{in} = 1$	D	H
0 0 0	Input Data	Input Data	A	A + 1	None	Circulate-Left with Carry
0 0 1	R1	R1	A + B	A + B + 1	R1	Circulate-Right without Carry
0 1 0	R2	R2	A + B′	A + B′ + 1	R2	No shift
0 1 1	R3	R3	A - 1	A	R3	0's to the output Bus
1 0 0	R4	R4	A OR B	A OR B	R4	–
1 0 1	R5	R5	A XOR B	A XOR B	R5	Shift Left with $I_L = 0$
1 1 0	R6	R6	A AND B	A AND B	R6	Shift Right with $I_R = 0$
1 1 1	R7	R7	A′	A′	R7	1's to the output Bus

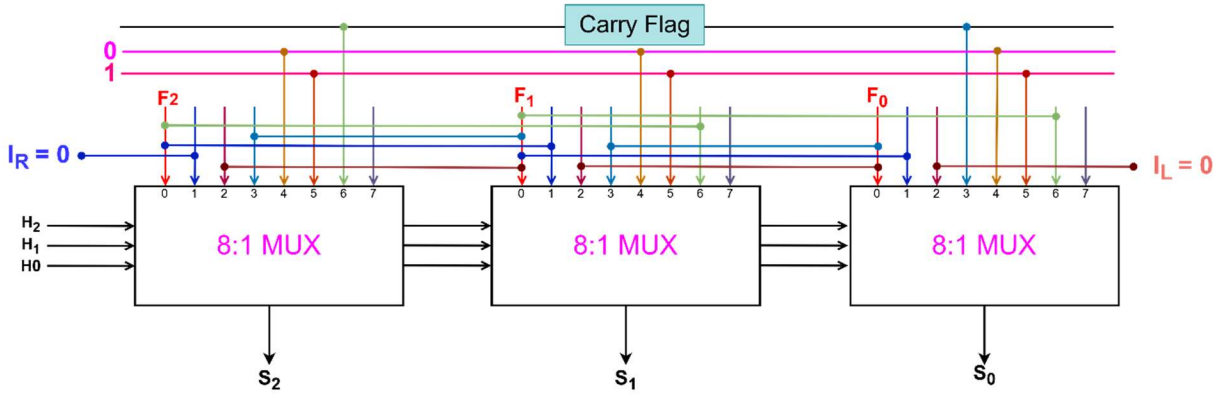
➔ ∴ **3-bit Shifter circuit:**



Problem1: Sketch a 3-bit shifter for the shift operations listed in Table.

Binary Code	Function of selection variables					
	A	B	D	F with $C_{in} = 0$	F with $C_{in} = 1$	H
0 0 0	Input Data	Input Data	None	A − 1	A, C ← 1	No Shift
0 0 1	R1	R1	R1	A + B	A + B + 1	Shift Right with $I_R = 0$
0 1 0	R2	R2	R2	A − B − 1	A − B	Shift Left with $I_L = 0$
0 1 1	R3	R3	R3	A, C ← 0	A + 1	Circulate Left with Carry
1 0 0	R4	R4	R4	A′	A′	0's to the output Bus
1 0 1	R5	R5	R5	A ∩ B	A ∩ B	1's to the output Bus
1 1 0	R6	R6	R6	A XOR B	A XOR B	Circulate Right with Carry
1 1 1	R7	R7	R7	A ∪ B	A ∪ B	-

➔ ∴ **3-bit Shifter circuit:**

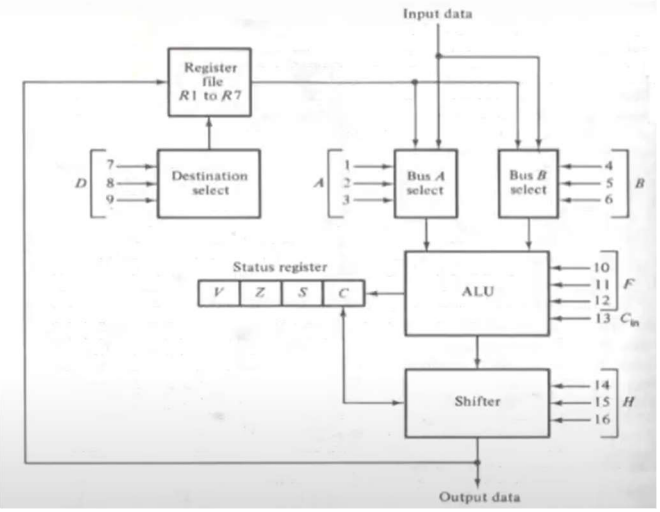


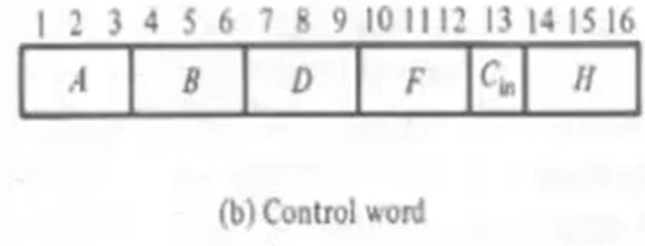
Design of a Processor Unit

1. **Selection Variables**: Control micro-operations in the processor during each clock pulse.
2. **Controlled Components**:
   - Buses
   - ALU (Arithmetic Logic Unit)
   - Shifter
   - Destination registers
3. **Processor Unit Structure**:
   - Seven general-purpose registers (R1 to R7)
   - Status register
   - Two multiplexers for selecting ALU inputs from the registers or external data
   - ALU output passes through a shifter
   - Shifter output can go to:
     - Any register
     - External output terminals
4. **Purpose**: To show how control variables determine micro-operations using a defined processor architecture.

- The control word is 16 bits, divided into 6 fields: A, B, D, F, H, and Cin.
- Each field except Cin is 3 bits.
- **A field (3 bits):** Selects source register for ALU left input.
- **B field (3 bits):** Selects source register for ALU right input.
- **D field (3 bits):** Selects destination register for ALU output.
- **F field (3 bits) + Cin (1 bit):** Selects the ALU function.
- **H field (3 bits):** Selects the shift operation type for the shifter unit.

Processor Unit with Control Variable





(b) Control word

Function of Controls Variables for the Processor Unit

Binary Code			Function of selection variables			
	B	A	D	F with $C_{in} = 0$	F with $C_{in} = 1$	H
0 0 0	Input Data	Input Data	None	A, C ← 0	A + 1	No shift
0 0 1	R1	R1	R1	A + B	A + B + 1	Shift Right with $I_R = 0$
0 1 0	R2	R2	R2	A - B - 1	A - B	Shift Left with $I_L = 0$
0 1 1	R3	R3	R3	A - 1	A, C ← 1	0's to the output Bus
1 0 0	R4	R4	R4	A OR B	–	–
1 0 1	R5	R5	R5	A XOR B	–	Circulate-Right with Carry
1 1 0	R6	R6	R6	A AND B	–	Circulate-Left with Carry
1 1 1	R7	R7	R7	A′	–	–

Table 2: 16-bit Control Word Sequence

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A			B			D			F			C_{in}	H		
Source 1			Source 2			Destination			Function			Carry	Shift		

Microoperations

Microoperation	Control word						Function
	A	B	D	F	Cin	H	
R1←R1−R2	001	010	001	010	1	000	Subtract R2 from R1
R3−R4	011	100	000	010	1	000	Compare R3 and R4
R5←R4	100	000	101	000	0	000	Transfer R4 to R5
R6←Input	000	000	110	000	0	000	Input data to R6
Output←R7	111	000	000	000	0	000	Output data from R7
R1←R1, C←0	001	000	001	000	0	000	Clear carry bit C
R3←shlR3	011	011	011	100	0	010	Shift-left R3 with IL=0
R1←crcR1	001	001	001	100	0	101	Circulate-right R1 with carry
R2←0	000	000	010	000	0	011	Clear R2

- To send a register's contents to the **Shifter** without altering the **carry bit**:
  - Use an **OR logic operation** in the ALU.
  - Select the **same register** for both **ALU inputs A and B**.
  - This keeps the value unchanged and avoids affecting the carry bit.

Problem2: **Write a 16-bit control word** for a micro-operation of **adding two numbers** stored in the **registers R1 and R2 with carry** and then storing the **results in the R5 register after shifting it to the right with no external data.**

Binary Code	B	A	D	F with $C_{in} = 0$	F with $C_{in} = 1$	H
0 0 0	Input Data	Input Data	None	A, C ← 0	A + 1	No shift
0 0 1	R1	R1	R1	A + B	A + B + 1	Shift Right with $I_R = 0$
0 1 0	R2	R2	R2	A - B - 1	A - B	Shift Left with $I_L = 0$
0 1 1	R3	R3	R3	A - 1	A, C ← 1	0's to the output Bus
1 0 0	R4	R4	R4	A OR B	–	–
1 0 1	R5	R5	R5	A XOR B	–	Circulate-Right with Carry
1 1 0	R6	R6	R6	A AND B	–	Circulate-Left with Carry
1 1 1	R7	R7	R7	A′	–	–

⇨ **Is given,**

Microoperation:

$R5 \leftarrow R1 + R2 + C_{in}$

So, Control Word Format:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0 0 1			0 1 0			1 0 1			0 0 1			1	1 0 1		
A			B			D			F			C_{in}	H		

In Binary form: 001 010 101 001 1 101b

In Hexadecimal form: 2A9Dh.

Problem3: For a processor unit with a 16-bit control word variable as in Table 2, **produce the control words** using Table 1 in **hexadecimal format** for the following listed micro-operations:

i. $R3 \leftarrow CRC\ R2$
ii. $R3 \leftarrow R1 + R4 + 1$
iii. $Output \leftarrow R6$
iv. $R3 \leftarrow R4 - R2$
v. $R2\ AND\ R4$

Table 1: For the Question

Binary Code	Function of selection variables					
	A	B	D	F with $C_{in} = 0$	F with $C_{in} = 1$	H
0 0 0	Input Data	Input Data	None	$A - 1$	$A, C \leftarrow 1$	No Shift
0 0 1	R1	R1	R1	$A + B$	$A + B + 1$	Shift Right with $I_R = 0$
0 1 0	R2	R2	R2	$A - B - 1$	$A - B$	Shift Left with $I_L = 0$
0 1 1	R3	R3	R3	$A, C \leftarrow 0$	$A + 1$	Circulate Left with Carry
1 0 0	R4	R4	R4	A'	A'	0's to the output Bus
1 0 1	R5	R5	R5	$A \cap B$	$A \cap B$	1's to the output Bus
1 1 0	R6	R6	R6	A XOR B	A XOR B	Circulate Right with Carry
1 1 1	R7	R7	R7	$A \cup B$	$A \cup B$	-

vi.

Table 2: 16-bit Control Word Sequence for the Question

1	2	3	4	5	6	7	8	9		10	11	12	13	14	15	16
A			B			D				F			C_{in}	H		
Source 1			Source 2			Destination				Function			Carry	Shift		

Microoperation:

Microoperation	Control word						Hexadecimal form
	A	B	D	F	Cin	H	
R3←CRC R2	010	010	011	111	0	110	$0x49F6$
R3←R1+R4+1	001	100	011	001	1	000	$0x3198$
Output ←R6	110	000	000	011	0	000	$0xC030$
R3←R4 - R2	100	010	011	010	1	000	$0x89A8$
R2 AND R4	010	100	000	101	0	000	$0x5050$

CONTROL LOGIC DESIGN MICRO-PROGRAM, FLOW-CHART, STATE-DIAGRAM

Introduction

- **Logic design** is a **complex** process.
- The **data processor** can be a **general-purpose processor** or a set of **registers** with **digital functions**.
- **Control logic** initiates all **micro-operations** in the processor.
- **Control logic** is a **sequential circuit** that generates **signals** to control and **sequence** operations.

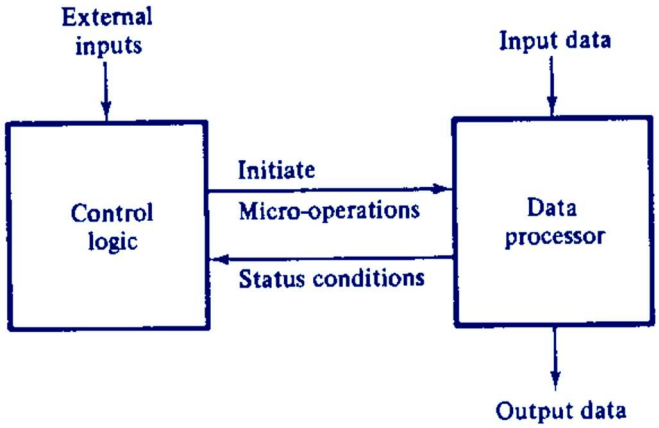


Fig. 10.1 Control logic and data processor interaction

Control Organization

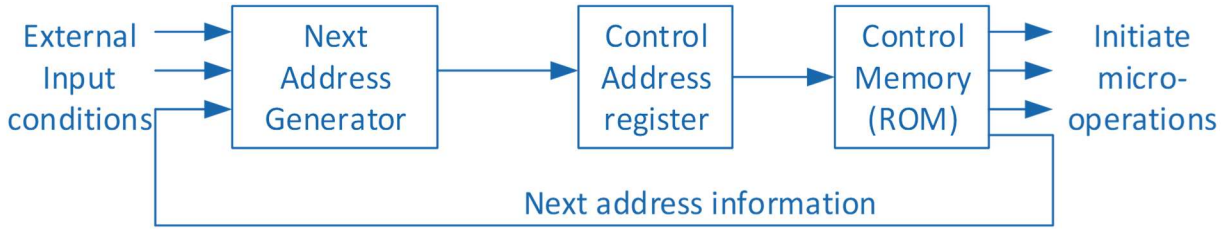
- **Methods of Control Organization**:
  - **One flip-flop per state** method
  - **Sequence register and decoder** method
  - **PLA control**
  - **Microprogram control**

- The **first two methods** use **SSI** and **MSI** circuits.
- **PLA** and **Microprogram control** use **LSI** devices.

Scale	Components Count	Year
SSI (Small Scale Integration)	< 100	1963
MSI (Medium Scale Integration)	100-1000	1970
LSI (Large Scale Integration)	1000-10000	1975
VLSI (Very Large Scale Integration)	$10000 - 10^6$	1980
ULSI (Ultra Large Scale Integration)	$> 10^6$	1990
GSI (Giga Scale Integration)	$> 10^{10}$	2010

Microprogram Control

- The **control unit** initiates **sequential microoperations**.
- A **Microprogrammed Control Unit (MCU)** stores **control variables** in **memory**.
- Each **control word** in memory is a **microinstruction**.
- A sequence of **microinstructions** forms a **microprogram**.



Hard-Wired Control - Example

- Design follows **five steps**:
  1. **Problem** is **stated**
  2. **Initial equipment configuration** is **assumed**
  3. **Algorithm** is **formulated**
  4. **Data processor** is **specified**
  5. **Control logic** is **designed**

1. Statement of the Problem

- We use **2's complement** to handle **negative binary numbers**.
- When doing **addition or subtraction** with **8-bit registers**, the result might be too big.
- If the result needs more than **8 bits**, it causes an **overflow**.
- The **extra bit** shows that an **overflow** happened.

A < B	$+6 \quad 0 \quad 000110$ $+9 \quad 0 \quad 001001$ (+) $+15 \quad 0 \quad 001111$	$-6 \quad 1 \quad 111010$ $+9 \quad 0 \quad 001001$ (+) $+3 \quad 0 \quad 000011$
A ≥ B	$+6 \quad 0 \quad 000110$ $-9 \quad 1 \quad 110111$ (+) $-3 \quad 1 \quad 111101$	$-9 \quad 1 \quad 110111$ $-9 \quad 1 \quad 110111$ (+) $-18 \quad 1 \quad 101110$

2. Equipment Configuration

- Two **signed binary numbers** have **n bits** each.
- Their **magnitudes** use **k = n – 1 bits** and are stored in **registers A and B**.
- The **sign bits** are stored in **flip-flops** named **As** and **Bs**.

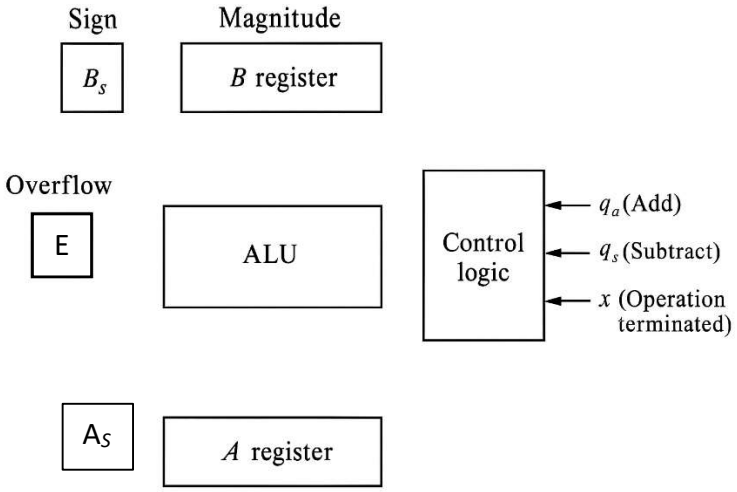


Fig. 10-6 Register and associated equipment configuration for the adder-subtractor

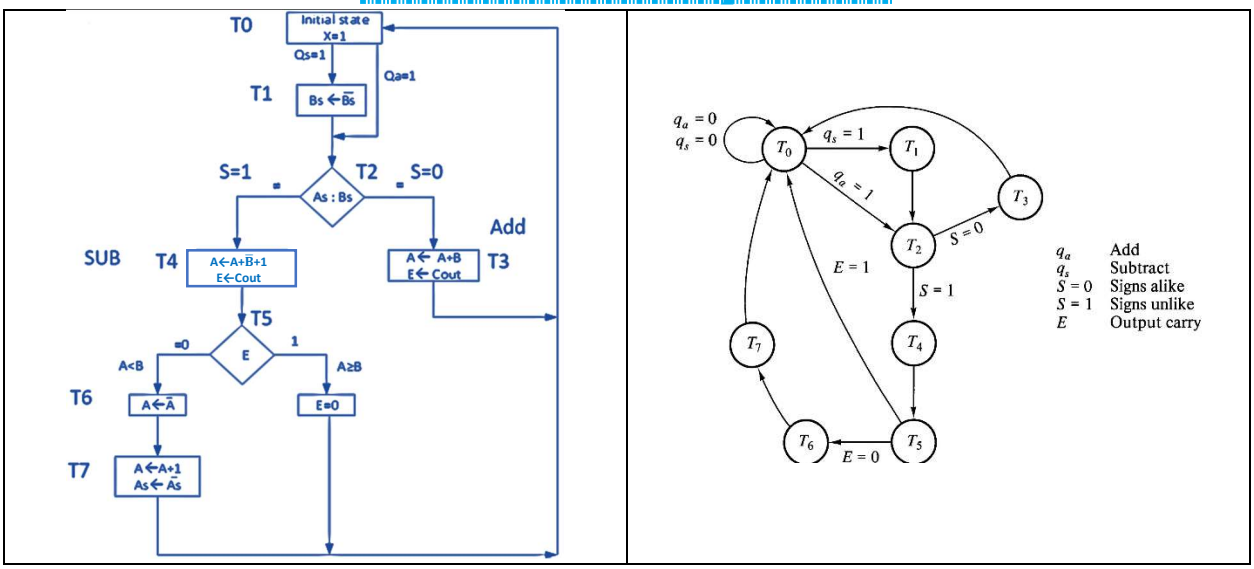
- The **ALU** does **arithmetic operations**.
- A **1-bit register E** stores the **overflow** (carry bit).
- **Registers A and A_s** hold the **result** and **sign**.
- **Control logic** uses input signals **q_a** (add) and **q_s** (subtract).
- **Output x** shows the **end of operation**.
- **Control logic** handles **inputs/outputs**, performs the right operation, and sets **x** when done.
- Final result is in **A**, **As**, and **overflow** in **E**.

3. Derivation of the Algorithm

- There are **8 ways** to add or subtract signed numbers: **(±A) ± (±B)**.
- To subtract, we **change B's sign** and then **add**.
- So, subtraction becomes just adding:
  - **(±A) − (+B) = (±A) + (−B)**
  - **(±A) − (−B) = (±A) + (+B)**
- This means we only need to think about **4 cases** of adding signed numbers: **(±A) + (±B)**.
- If **A and B have the same sign**, just **add** their values, and keep that sign.
- If **A and B have different signs**, **subtract** the smaller value from the bigger one, and the result takes the **sign of the bigger number**.

Operations	**ADD Magnitudes (Signs are equal)**	**Subtract Magnitudes (Signs are Different)**	
		If A ≥ B	***If A < B***
$(+\mathbf{A}) + (+\mathbf{B})$	$+(\mathbf{A}+\mathbf{B})$	−	−
$(+\mathbf{A}) + (-\mathbf{B})$	−	$+(\mathbf{A}-\mathbf{B})$	$-(\mathbf{B}-\mathbf{A})$
$(-\mathbf{A}) + (+\mathbf{B})$	−	$-(\mathbf{A}-\mathbf{B})$	$+(\mathbf{B}-\mathbf{A})$
$(-\mathbf{A}) + (-\mathbf{B})$	$-(\mathbf{A}+\mathbf{B})$	−	−

Flowchart and state diagram for sign magnitude addition and subtraction operation



1. The operation starts when **qa (add)** or **qs (subtract)** is given.
2. If **qs**, then it's **subtraction**, so **flip B's sign**.
3. If **qa**, then it's **addition**, so **keep B's sign**.
4. Compare the **signs of A and B (As and Bs)**:
    - If **same**, follow the **'=' path (S =0)**
    - If **different**, follow the **'≠' path (S=1)**

➢ If signs are the same:

- Do **A + B**, put the result in **A**
- If there's a **carry**, store it in **E (overflow flip-flop)**

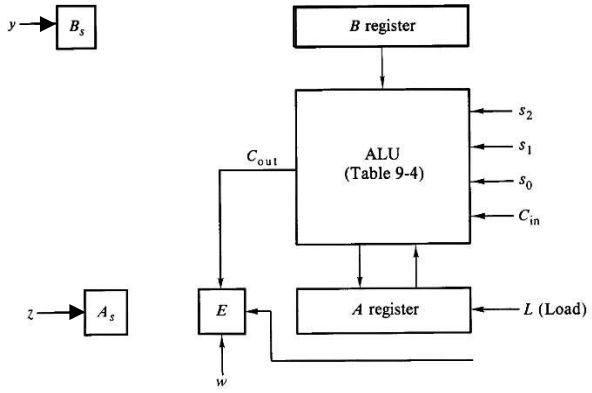
➢ If signs are different:

- Do **A - B** (by adding **2's complement of B**)
- If **E = 1**, then **A ≥ B**, and result is **correct**
- If **E = 0**, then **A < B**
    - Take **2's complement of A** (do **A′ + 1**)
    - **Flip the sign (As)**

5. At the end:

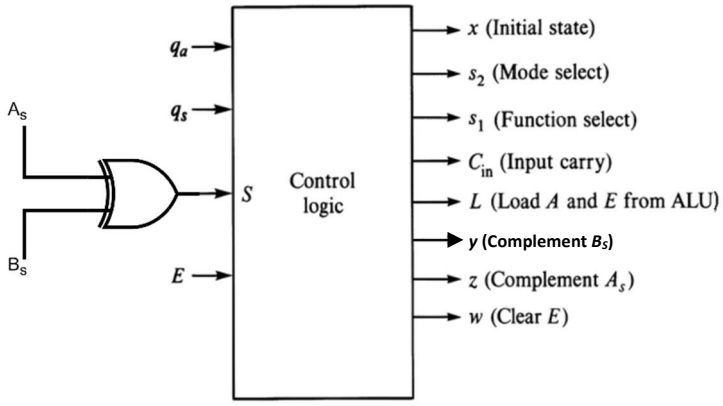
- Go back to the **start**
- Set **x = 1** to show operation is **done**
- If the result's sign is still like **As**, don't change it

4. Data Processor Register



- All operations between **A and B** are done by the **ALU**.
- Operations on **As**, **Bs**, and **E** need **separate control signals** to start.

5. Control Block Diagram



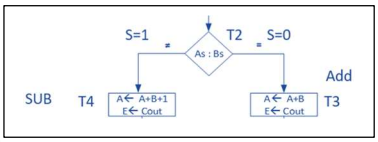
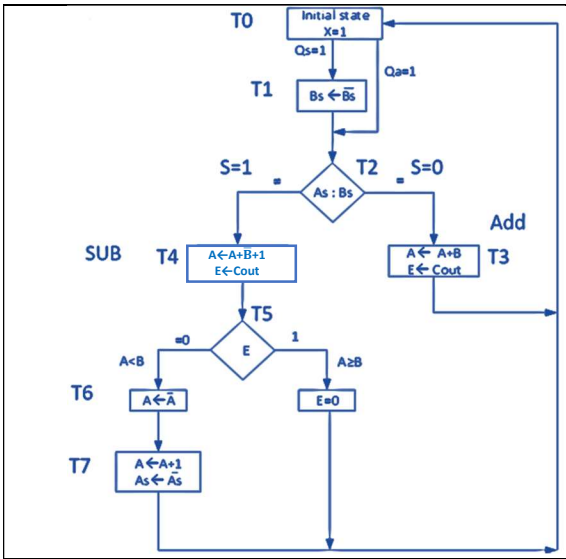


Fig. 10-8 (b) Control Logic Block Diagram

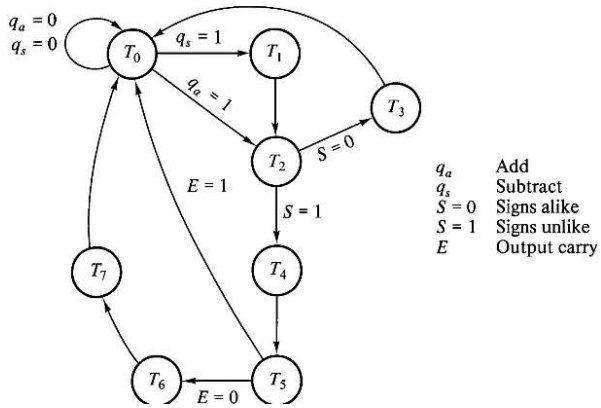
- The **control logic** gets **5 inputs**:
  - **2** from the **external environment**
  - **3** from the **data processor**
- To make things easier, a new variable **S** is defined as:

  ≫S = A_s XOR B_s

Control State Diagram



- $\gg T_0$: Initial State $x = 1$
- $\gg T_1: B_s \leftarrow \overline{B}_s$
- $\gg T_2$: Nothing
- $\gg T_3: A \leftarrow A + B, \quad E \leftarrow C_{out}$
- $\gg T_4: A \leftarrow A + \overline{B} + 1, \quad E \leftarrow C_{out}$
- $\gg T_5: E \leftarrow 0$
- $\gg T_6: A \leftarrow \overline{A}$
- $\gg T_7: A \leftarrow A + 1, \quad A_s \leftarrow \overline{A}_s$



- $\gg T_0$: Initial State $x = 1$
- $\gg T_1: B_s \leftarrow \overline{B}_s$
- $\gg T_2$: Nothing
- $\gg T_3: A \leftarrow A + B, \quad E \leftarrow C_{out}$
- $\gg T_4: A \leftarrow A + \overline{B} + 1, \quad E \leftarrow C_{out}$
- $\gg T_5: E \leftarrow 0$
- $\gg T_6: A \leftarrow \overline{A}$
- $\gg T_7: A \leftarrow A + 1, \quad A_s \leftarrow \overline{A}_s$

- Start with an initial state called **T0**.
- Then move to other states: **T1, T2, T3**, etc.
- For each state, decide what **micro-operations** the **control logic** should do.
- This helps create the **state diagram** and a list of **register-transfer operations** for each state.

Sequence of Register Transfers

Binary Code			Function of selection variables			
	B	A	F with $C_{in} = 0$	F with $C_{in} = 1$	D	H
0 0 0	Input Data	Input Data	A	A + 1	None	Circulate-Left with Carry
0 0 1	R1	R1	A + B	A + B + 1	R1	Circulate-Right without Carry
0 1 0	R2	R2	A + B′	A + B′ + 1	R2	No shift
0 1 1	R3	R3	A - 1	A	R3	0's to the output Bus
1 0 0	R4	R4	A OR B	A OR B	R4	–
1 0 1	R5	R5	A XOR B	A XOR B	R5	Shift Left with $I_L = 0$
1 1 0	R6	R6	A AND B	A AND B	R6	Shift Right with $I_R = 0$
1 1 1	R7	R7	A′	A′	R7	1's to the output Bus

Control State	Control Words/Outputs									Symbol Definitions
	x	S_2	S_1	S_0	C_{in}	L	y	z	w	
$\gg T_0$: Initial State $x = 1$	1	0	0	0	0	0	0	0	0	x = Initial state
$\gg T_1: B_s \leftarrow \overline{B}_s$	0	0	0	0	0	0	1	0	0	s_2 = Mode selects
$\gg T_2$: Nothing	0	0	0	0	0	0	0	0	0	s_1 = Function selects
$\gg T_3: A \leftarrow A + B,\ E \leftarrow C_{out}$	0	0	0	1	0	1	0	0	0	s_0 = Function selects
$\gg T_4: A \leftarrow A + \overline{B} + 1, E \leftarrow C_{out}$	0	0	1	0	1	1	0	0	0	C_{in} = Input carry
$\gg T_5: E \leftarrow 0$	0	0	0	0	0	0	0	0	1	L = Load A and E from ALU
$\gg T_6: A \leftarrow \overline{A}$	0	1	1	1	0	1	0	0	0	y = Complement B_s
$\gg T_7: A \leftarrow A + 1,\ A_s \leftarrow \overline{A}_s$	0	0	0	0	1	1	0	1	0	z = Complement A_s
										w = Clear E

Sequence of Register Transfers

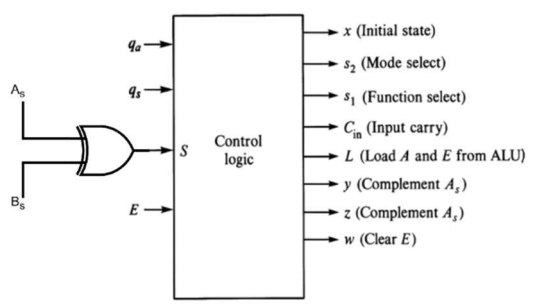


Fig. 10-8 (b) Control Logic Block Diagram

$\gg x = T_0$

$\gg S_2 = T_6$

$\gg S_1 = T_4 + T_6$

$\gg S_0 = T_3 + T_6$

$\gg C_{in} = T_4 + T_7$

$\gg L = T_3 + T_4 + T_6 + T_7$

$\gg y = T_1$

$\gg z = T_7$

$\gg w = T_5$

Logic Equations for Hardwired Control Unit
Boolean Functions for Output Control

Hardware Configuration

ROM Bit 13	ROM Bit 14	MUX Select Function
0	0	Increment CAR
0	1	Load input to CAR
1	0	Load inputs to CAR if **S = 1**, increment CAR if **S = 0**
1	1	Load inputs to CAR if **E = 1**, increment CAR if **E = 0**

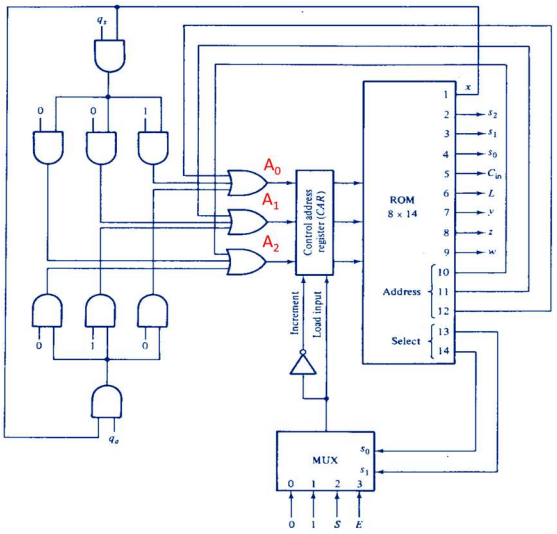


Fig. 10-10 Organization of the microprogram control unit

- ⇨ The **control memory** is a **ROM** with **8 words**, each **14 bits** long (1 word = two 8-bit bytes = 16 bits usually).
- ⇨ The first **9 bits** of a micro-instruction are for **control signals** (to trigger micro-operations).
- ⇨ The last **5 bits** help choose the **next address**.

Registers:

- **CAR (Control Address Register)** holds the address for control memory.
  - If **load is enabled**, CAR loads a new value.
  - If not, it just **increments by 1** (works like a counter with parallel load).

Bits:

- **Bits 10–12**: Hold the **next address** for CAR.
- **Bits 13–14**: Control a **MUX (Multiplexer)** that decides what goes into CAR.
- **Bit 1**: Called **x**, helps determine the **initial state**.
  - If **x = 1**, address must be **000**.
    - If q_s **= 1**, load **address 001** into CAR.
    - If q_a **= 1**, load **address 010** into CAR.
    - If **both are 0**, use **bits 10–12** for CAR address.

So, in short:

- The **control memory** tells what to do and where to go next.
- The **CAR** decides which instruction to fetch from control memory.
- The **MUX**, **x**, q_s, q_a, and **S** all help decide the **next step** of the controller.

Hardware Configuration

ROM Bit 13	ROM Bit 14	MUX Select Function
0	0	Increment CAR
0	1	Load input to CAR
1	0	Load inputs to CAR if **S = 1**, increment CAR if **S = 0**
1	1	Load inputs to CAR if **E = 1**, increment CAR if **E = 0**

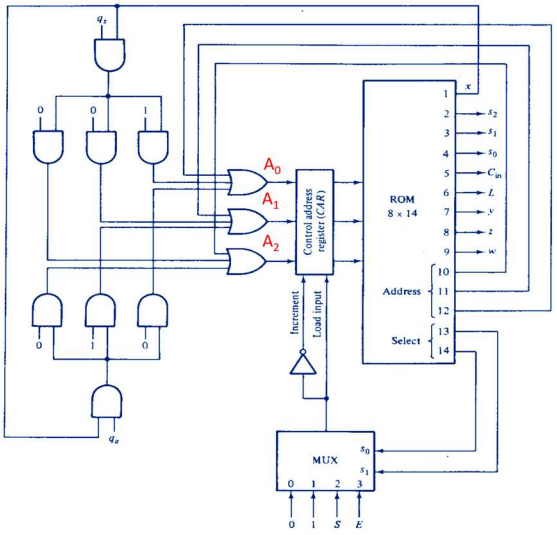


Fig. 10-10 Organization of the microprogram control unit

⇨ When **bits 13 and 14 = 11**, the status variable **E** is selected.
- If **E = 1**, the **address** in the micro-instruction is **loaded** into **CAR**.
- If **E = 0**, the **CAR is incremented**.
- This lets control **choose between two addresses**, based on the **status bit**.

Microprogramming:

- After designing the **microprogrammed control unit**, the next step is to write the **micro-code** for **control memory**.
- This process is called **microprogramming** — deciding the **bit values** for all **8 words** in the memory (each 14 bits long).

Writing Microprograms:

- Use the **register-transfer method** to write the microprogram.
- Each instruction is written as a **symbolic register-transfer statement**, not in 1s and 0s.
- Each symbolic statement has:
  - The **address** where it will be stored in memory.
  - A **register-transfer operation**.
  - Optional **comments** to clarify what's happening.

Conditional Branching:

- You can use **conditional control statements** to tell the controller **where to go next**, based on **status bits** like **S** or **E**.

Summary:

- Think in terms of **register operations** (symbols), not bits.
- Once symbolic microprogram is done, you can **translate** it into **binary micro-instructions**.
- An example of a symbolic microprogram for **Adder-Subtractor** is shown in **Table 10-2**, with:
  - **Address**
  - **Instruction**
  - **Comments**

Microprogram for Control Memory

ROM Address	Microinstruction	Comments:
0	$x = 1$, if $(qx = 1)$ then (go to 1), if $(qa = 1)$ then (go to 2), if $(qs \wedge qa = 0)$ then (go to 0)	Load 0 or external address
1	$\mathbf{Bs \leftarrow \overline{B}s}$	$q_s = 1$, start subtraction
2	If $(\mathbf{S = 1})$ then (go to 4)	$q_a = 1$, start addition
3	$\mathbf{A \leftarrow A + B}$, $\mathbf{E \leftarrow Cout}$, go to 0	Add magnitudes and return
4	$\mathbf{A \leftarrow A + \overline{B} + 1}$, $\mathbf{E \leftarrow Cout}$	Subtract magnitudes
5	If $(\mathbf{E = 1})$ then (go to 0), $\mathbf{E \leftarrow 0}$	Operation terminated if E=1
6	$\mathbf{A \leftarrow \overline{A}}$	E=0, complement A
7	$\mathbf{A \leftarrow A + 1}$, $\mathbf{As \leftarrow \overline{A}s}$, go to 0	Done, return to address 0

Binary microprogram for control memory

ROM address	x	s_2	s_1	s_0	C_{in}	L	y	z	w	Address			Select	
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
0 0 0	1	0	0	0	0	0	0	0	0	0	0	0	0	1
0 0 1	0	0	0	0	0	0	1	0	0	0	1	0	0	1
0 1 0	0	0	0	0	0	0	0	0	0	1	0	0	1	0
0 1 1	0	0	0	1	0	1	0	0	0	0	0	0	0	1
1 0 0	0	0	1	0	1	1	0	0	0	1	0	1	0	1
1 0 1	0	0	0	0	0	0	0	0	1	0	0	0	1	1
1 1 0	0	1	1	1	0	1	0	0	0	1	1	1	0	1
1 1 1	0	0	0	0	1	1	0	1	0	0	0	0	0	1

Binary Micro-Program (Table 10-3):

- The table shows the **binary version** of the **micro-program**.
- The **first column** lists the **8 binary addresses** of the **ROM**.
- The **second column** shows the **micro-instructions** in **binary/machine language**.
- The **third column** shows the actual **ROM contents** (binary) used to **program the ROM**.

Structure of Each ROM Word:

- Each word in the ROM is **14 bits** long:
  - The **first 9 bits** are the **control word** (these trigger the **micro-operations**, as seen in **Fig. 10-9(b)**).
  - The **last 5 bits** are for **control flow**, taken from the **conditional control statements** in the symbolic micro-program.

Summary:

- **Table 10-3** converts the symbolic program into a **machine-readable format**.
- These binary values are what get **loaded into the ROM** to make the control unit work.

Hardware Configuration

- **Address 0** is the **initial state**. It produces an output **x = 1**.
- The **next address** depends on the external inputs: **qs** and **qa**.
  - If **qs = 1**, control **goes to address 1**
  - If **qa = 1**, control **goes to address 2**
  - If **both qs and qa = 0**, control **stays at address 0** (keeps repeating the same instruction)

These decisions are made using **conditional control statements** with **"go to"** commands.

Conditional control statements

- **Conditional control statements** in the other micro-instructions use **status variables S and E.**
- A **"go to" statement without any condition** means an **unconditional jump** to the specified address.

  Example: **go to 0** → move to **address 0** no matter what.
- If **there is no "go to"** in a micro-instruction, control automatically moves to the **next address in order**.
- If an **"if" condition is false**, the next instruction is simply taken from the **next address**.
- The micro-instructions at the **8 addresses** come directly from the **control specifications** (Fig. 10-9). They match exactly with the instructions in **Fig. 10-9(b)**.
- The **conditional control statements** define the **sequence of addresses**, just like a **state diagram** (Fig. 10-9(a)).

Example of Micro-program

What is a Macro-operation?

A **macro-operation** is a **high-level task**, like **ADD**, **SUB**, or **LOAD**.
To carry it out, the system runs a **sequence of small steps** called **micro-instructions**.

What is a Micro-program?

A **micro-program** is a **set of micro-instructions** stored in **control memory**.
Together, they form a **routine** that performs the full **macro-operation**.

How it Works:

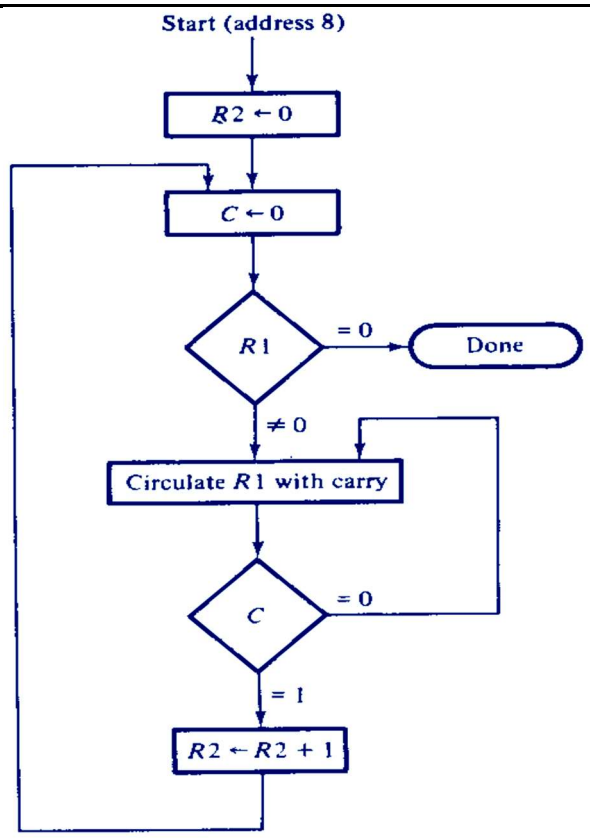
1. The **macro-operation** is **started** when an **external address** (from outside the control unit) sends the **first control memory address**.
2. The **micro-program** begins executing from that address.
3. It continues running each micro-instruction in order.
4. When the **task is finished**, the **last micro-instruction** in the routine **loads a new external address**.
5. That new address starts the **next macro-operation**.

Flow chart for counting the number of 1's in register R1

Problem1: Prepare a flow chart that will **count the number of 1's** in **register, R1** and **then store the counts in register R2. Determine the outputs of the R2** and **R1 registers** as well as of the **carry flag** after each clock cycle or timing state. Also, write the corresponding **symbolic microprogram**.

	R1								C	R2
Timing States	0	0	1	1	0	1	0	1	0	0
T1										
T2										
T3										
T4										
T5										
T6										
T7										
T8										

Flow chart



➔ **We want to:**

- Count how many **1s** are present in **R1**.
- Store the **count** into **R2**.

➔ **Idea Behind the Micro-program:**

We'll do this step by step using simple micro-operations:

1. **Clear R2** (Address 8) → Start with count = 0
2. **Clear carry (C)** (Address 9)
3. **Check each bit of R1** from right to left:
   - If the bit is **1**, increment R2.
4. Repeat this until all 8 bits of R1 are checked.
5. Done!

➔ **Registers & Tools:**

We assume:

≫**R1**: Input register (holds the original 8-bit number)
≫**R2**: Output register (holds the count of 1s)
≫**E**: Used as a flag
≫**ALU**: Performs basic operations like shift and increment
≫A **counter or loop** can be handled via the control unit state transitions

➔ **How Loop Works:** (Address 10 to 13)	**Final Result:**
We repeat the **check** + **shift** operations 8 times (for 8 bits). Each time we: ≫ Check if LSB bit of R1 is 1? ≫ Write 1 to C if it is, else write 0 ≫ Increment 1 to R2 if C is 1. ≫ Then shift R1 to the right to check next bit	At the end of the micro-program: • **R2** contains the number of 1s in R1

Now fill the table:

Timing States	R1 (Binary)								C	R2(D)
	0	0	1	1	0	1	0	1	0	0
T1	0	0	0	1	1	0	1	0	1	1
T2	0	0	0	0	1	1	0	1	0	1
T3	0	0	0	0	0	1	1	0	1	2
T4	0	0	0	0	0	0	1	1	0	2
T5	0	0	0	0	0	0	0	1	1	3
T6	0	0	0	0	0	0	0	0	1	4
T7	0	0	0	0	0	0	0	0	0	4

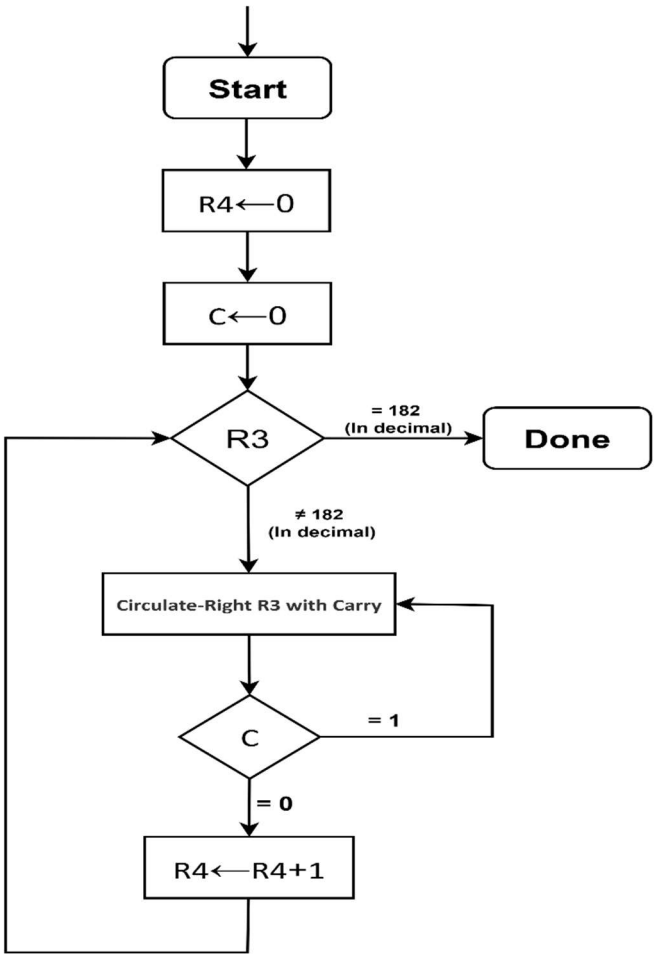
Corresponding symbolic microprogram.:

ROM Address	Microinstruction	Comments
8	R2 ← 0	Clear R2 counter
9	R1 ← R1, C ← 0	Clear C, set status bits
10	If (Z = 1) then (go to external address)	Done if R1 = 0
11	R1 ← crc R1	Circulate R1 right with carry
12	If (C = 0) then (go to 11)	Circulate again if C = 0
13	R2 ← R2 + 1, go to 9	Carry = 1, increment R2

Problem2: **Prepare a flow chart** that will **count the number of 0's in register, R3** and then **store the counts in register R4**. **Determine the outputs of the R4 (in binary) and R3 (in decimal) registers** as well as of the **carry flag** after each clock cycle or timing state. **Determine the number of states** that are required to complete the operation.

Timing States	R3								C	R4
	0	**1**	**0**	**1**	**1**	**0**	**1**	**1**	**0**	**0**
T1										
T2										
T3										
T4										
T5										
T6										
T7										
T8										

Flow chart



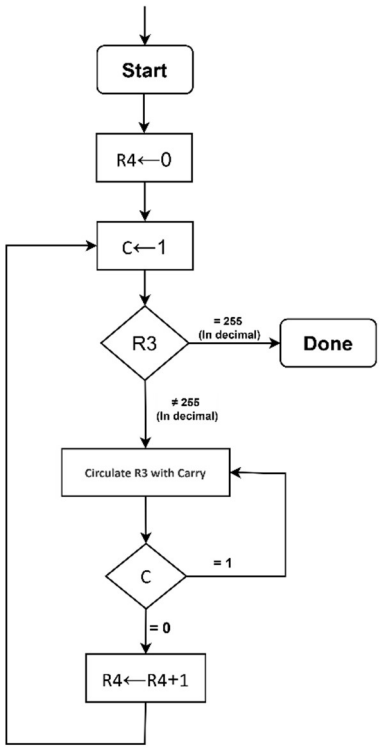
Now fill the table:

Timing States	R3 (Binary)								C	R3(D)	R4(D)	R4(B)
	0	1	0	1	1	0	1	1	0	91	0	0000 0000
T1	0	0	1	0	1	1	0	1	1	45	0	0000 0000
T2	1	0	0	1	0	1	1	0	1	150	0	0000 0000
T3	1	1	0	0	1	0	1	1	0	203	1	0000 0001
T4	0	1	1	0	0	1	0	1	1	101	1	0000 0001
T5	1	0	1	1	0	0	1	0	1	178	1	0000 0001
T6	1	1	0	1	1	0	0	1	0	217	2	0000 0010
T7	0	1	1	0	1	1	0	0	1	108	2	0000 0010
T8	1	0	1	1	0	1	1	0	0	182	3	0000 0011

∴ **Final Outputs:**

- ⇨ **R4 = 00000011 = 3** (→ 3 zeroes found)
- ⇨ **R3 = 10110110 = 182**
- ⇨ **C = 0**
- ⇨ **Number of states** = 8

Or



Now fill the table:

Timing States	R3 (Binary)								C	R3(D)	R4(D)	R4(B)
	0	1	0	1	1	0	1	1	0	91	0	0000 0000
T1	1	0	1	0	1	1	0	1	1	173	0	0000 0000
T2	1	1	0	1	0	1	1	0	1	214	0	0000 0000
T3	1	1	1	0	1	0	1	1	0	235	1	0000 0001
T4	1	1	1	1	0	1	0	1	1	245	1	0000 0001
T5	1	1	1	1	1	0	1	0	1	250	1	0000 0001
T6	1	1	1	1	1	1	0	1	0	253	2	0000 0010
T7	1	1	1	1	1	1	1	0	1	254	2	0000 0010
T8	1	1	1	1	1	1	1	1	0	255	3	0000 0011

∴ Final Outputs:

- **R4 = 00000011 = 3** (→ 3 zeroes found)
- **R3 = 11111111 = 255**
- **C = 1**
- **Number of states** = 8